ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA
CAMPUS DI CESENA

Programmazione di Reti
Laboratorio #3

Andrea Piroddi

Dipartimento di Informatica, Scienza e Ingegneria

SOCKET TCP

Programmazione di un Socket TCP
Web Server

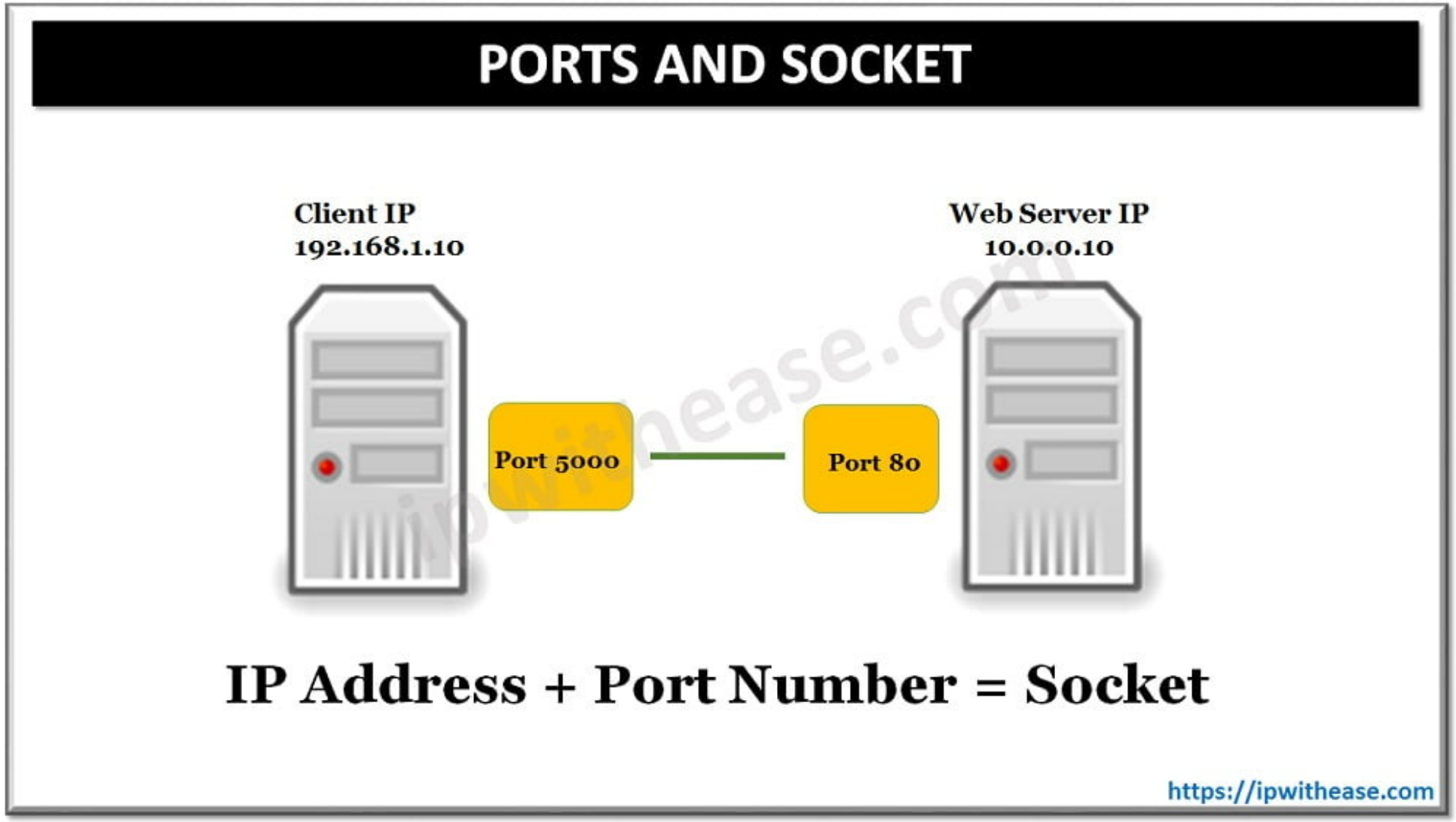
ALMA MATER STUDIORUM
UNIVERSITÀ DI BOLOGNA
CAMPUS DI CESENA

Socket

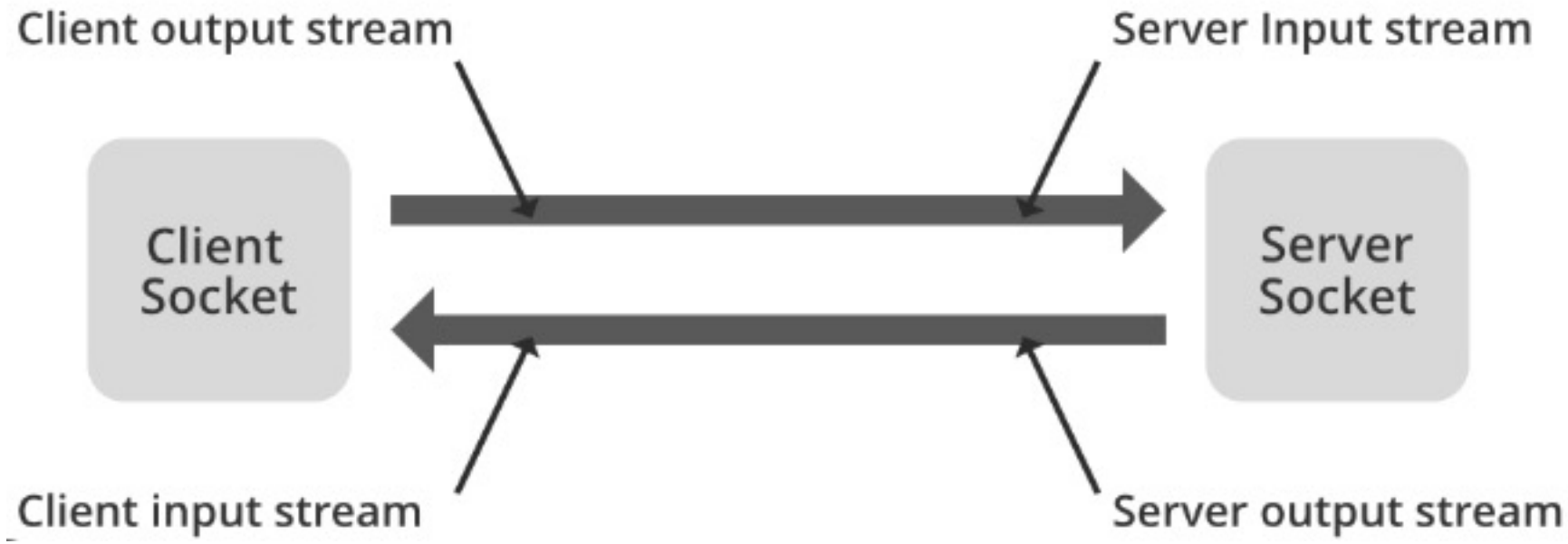
Un **socket** è un ***oggetto software*** che **permette l'invio e la ricezione di dati, tra host remoti** (tramite una rete) **o tra processi locali.**

Più precisamente, il concetto di **socket** si basa sul modello Input/Output su file di Unix, quindi sulle operazioni di ***open***, ***read***, ***write*** e ***close***; l'utilizzo, infatti, avviene secondo le stesse modalità, aggiungendo i parametri utili alla comunicazione, quali **indirizzi ip**, **numeri di porta** e **protocolli**.

Socket


PORTS AND SOCKET
Client IP
192.168.1.10
Web Server IP
10.0.0.10
Port 5000
Port 80
IP Address + Port Number = Socket
https://ipwithease.com

Socket



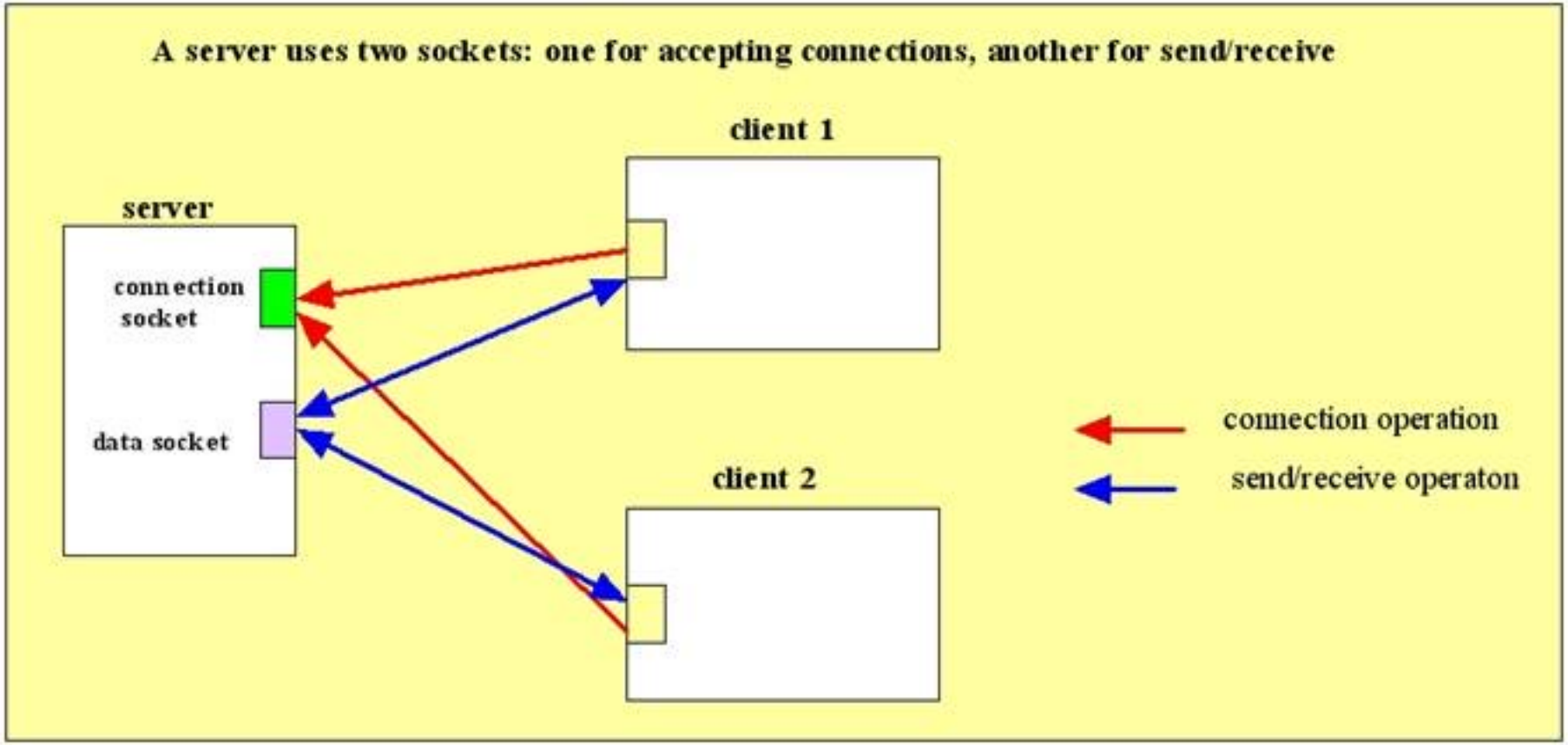
Socket locali e remoti in comunicazione formano una **coppia** (pair), composta da **indirizzo** e **porta** di client e server; tra di loro c'è una connessione logica.

Socket

I sistemi operativi offrono delle **API (Application Programming Interface)** che permettono ai programmi di **creare, gestire e utilizzare** i socket di rete per la comunicazione tra dispositivi.

- Le applicazioni non accedono direttamente all'hardware di rete, ma usano le **API del sistema operativo** per inviare e ricevere dati tramite i socket.
- Queste API forniscono funzioni standardizzate per creare un socket, connettersi a un server, inviare dati e chiudere la connessione.

Socket



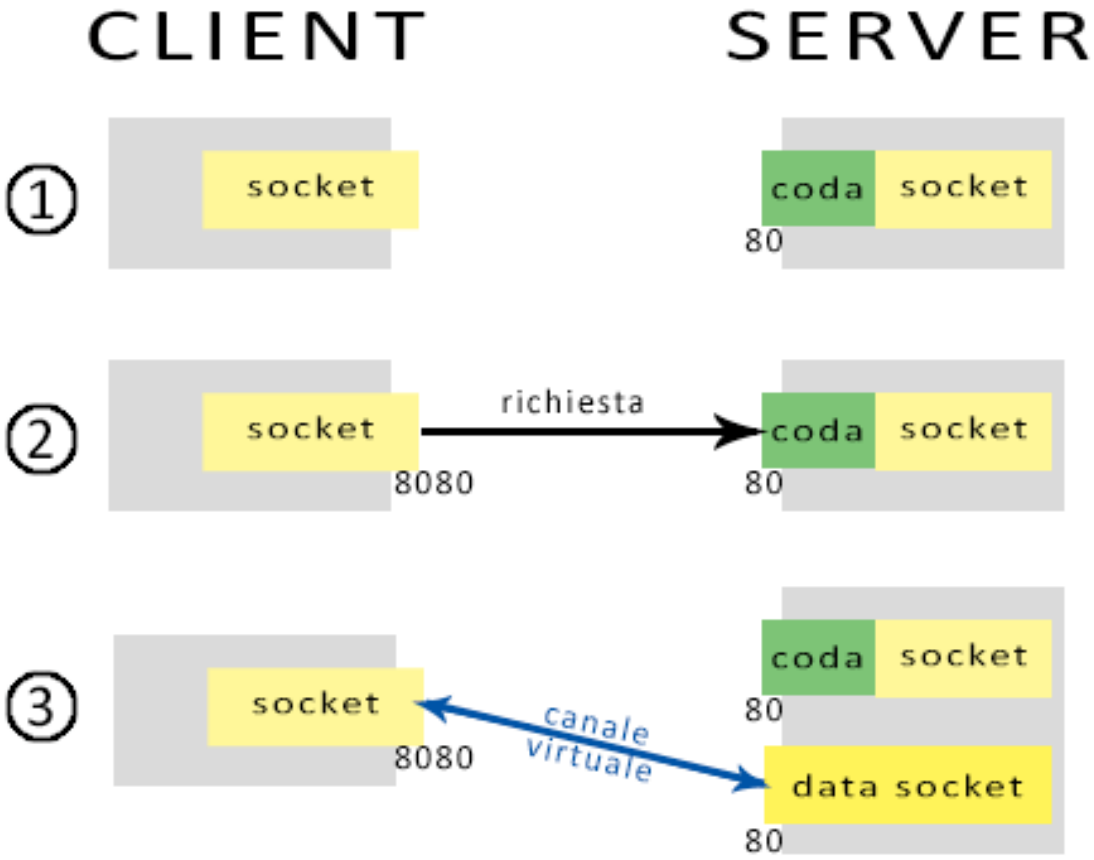
Esempi di API per socket nei diversi sistemi operativi:

- **Windows:** Usa la libreria **Winsock** (Windows Sockets API).
- **Linux/macOS:** Usa le **Berkeley Sockets API** (socket.h).
- **Python:** Usa il modulo **socket**, che funge da interfaccia alle API native del sistema operativo.

CREAZIONE di un SOCKET

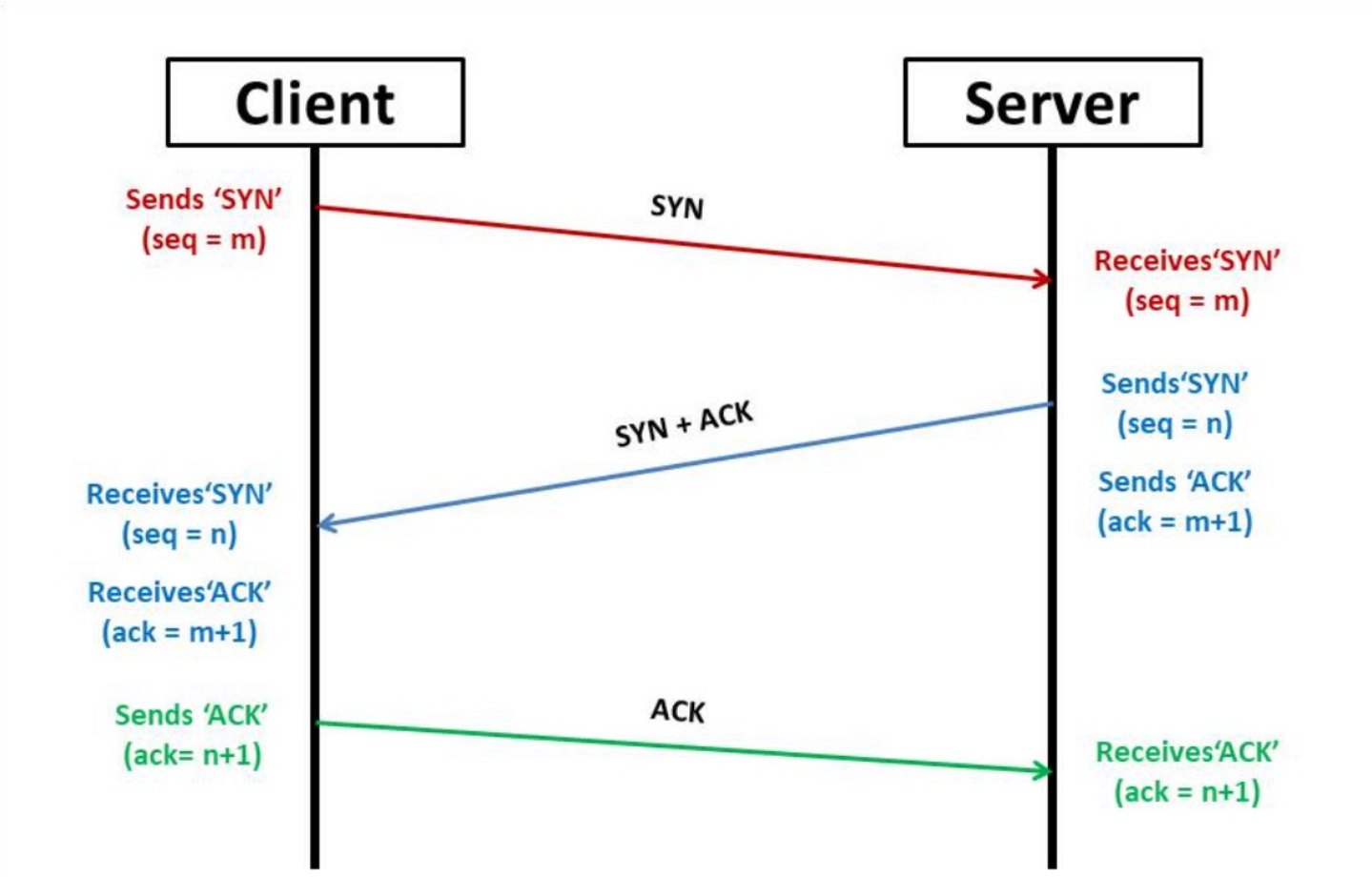
1. Creazione dei socket

Client e **server** creano i loro rispettivi **socket**, e il **server** lo pone in **ascolto** su una **porta**. Dato che il server può creare più connessioni con client diversi (ma anche con lo stesso), ha bisogno di una **coda** per gestire le varie richieste.



2. Richiesta di connessione

Il **client** effettua una **richiesta di connessione** verso il server. Il **server** riceve la richiesta e, nel caso in cui sia accettata, viene creata una **nuova connessione**.



3-Way Handshaking(for establishing connection)

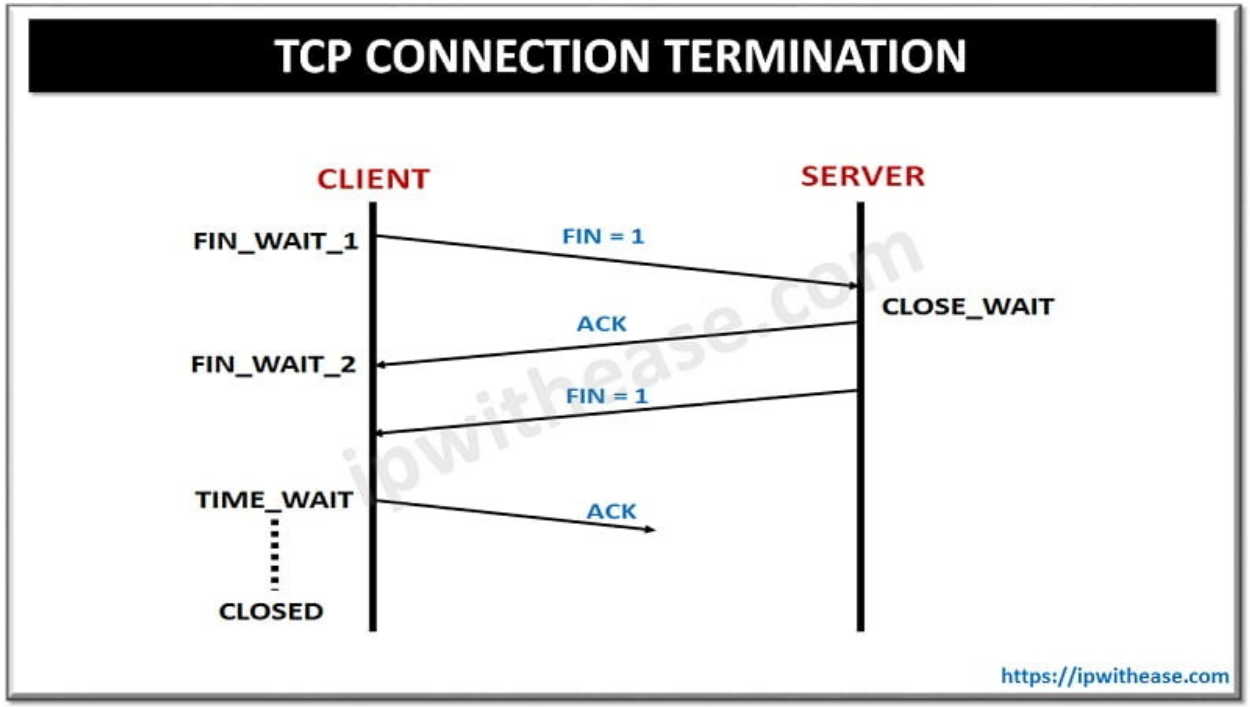
3. Comunicazione

Ora **client** e **server** comunicano attraverso un **canale virtuale**, tra il socket del primo, ed uno nuovo del server, creato appositamente per il flusso dei dati di questa connessione: **data socket**.

Coerentemente a quanto avete visto nella teoria, il server crea il data socket perchè il primo serve esclusivamente alla gestione delle richieste. È possibile, quindi, che ci siano molti client a comunicare con il server, ciascuno verso il **data socket** creato dal server per loro.

4. Chiusura della connessione

Essendo il TCP un protocollo orientato alla connessione, quando non si ha più la necessità di comunicare, il client lo comunica al server, che ne **deistanzia il data socket**.



La connessione viene così chiusa.

Famiglie di socket

I tipi di **protocolli** utilizzati dal **socket**, ne definiscono la **famiglia** (o dominio).

Possiamo distinguere, ad esempio, due importanti famiglie:

- ***AF_INET***: comunicazione tra host remoti, tramite Internet;
- ***AF_UNIX***: comunicazione tra processi locali, su macchine Unix. Questa famiglia è anche chiamata *Unix Domain Socket*.

Esercizio 1: Programmazione di un Socket - Web Server

In questo laboratorio vediamo le basi della programmazione dei socket per le connessioni TCP in Python:

- come creare un socket
- associarlo a un indirizzo e una porta specifici,
- inviare e ricevere un pacchetto HTTP.

Esercizio 1: Programmazione di un Socket - Web Server

Sviluppiamo un server web che gestisce una richiesta HTTP alla volta.

- Il server Web deve accettare e analizzare la richiesta HTTP
- ottenere il file richiesto dal file system del server
- creare un messaggio di risposta HTTP costituito dal file richiesto preceduto da righe di intestazione e quindi inviare la risposta direttamente al client.
- Se il file richiesto non è presente nel server, il server deve inviare un messaggio **HTTP "404 non trovato"** al client.

Esercizio 1: Programmazione di un Socket - Web Server

Su IOL trovate nel Laboratorio il codice Python: «**TCP_Socket_Server.py**» e un file «**index.html**».

Scaricate i due file (***index.html*** e ***TCP_Socket_Server.py***) sul vostro PC e metteteli nella stessa directory.

Esercizio 1: Programmazione di un Socket - Web Server

Aprite il file Python e osservate il valore di porta all'interno del file python (esempio **serverPort=8080**)

```python
# Corso di Programmazione di Reti - Laboratorio - UniversitÃ  di Bologna
# Socket_Programming_Assignment - WebServer - F. Callegati - G.Pau - A. Piroddi

import sys
from socket import *
serverPort=8080
serverSocket = socket(AF_INET, SOCK_STREAM)
server_address=('localhost',serverPort)
serverSocket.bind(server_address)

#listen(1) Definisce la lunghezza della coda di backlog, ovvero il numero
#di connessioni in entrata che sono state completate dallo stack TCP / IP
#ma non ancora accettate dall'applicazione.
serverSocket.listen(1)
print ('the web server is up on port:',serverPort)

while True:

    print ('Ready to serve...')
    connectionSocket, addr = serverSocket.accept()
    print(connectionSocket,addr)

    try:

        message = connectionSocket.recv(1024)
        if len(message.split())>0:
            print (message,'::',message.split()[0],':',message.split()[1])
            filename = message.split()[1]
            print (filename,'//',filename[1:])
            f = open(filename[1:],'r+')
            outputdata = f.read()
            print (outputdata)

     #Invia la riga di intestazione HTTP nel socket con il messaggio OK

            connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())
            connectionSocket.send(outputdata.encode())
            connectionSocket.send("\r\n".encode())
            connectionSocket.close()


    except IOError:
 #Invia messaggio di risposta per file non trovato
        connectionSocket.send(bytes("HTTP/1.1 404 Not Found\r\n\r\n","UTF-8"))
        connectionSocket.send(bytes("<html><head></head><body><h1>404 Not Found</h1></body></html>\r\n","UTF-8"))
        connectionSocket.close()
```

Esercizio 1: Programmazione di un Socket - Web Server

ed eseguite il codice

```
In [2]: runfile('/Users/apirodd/OneDrive - Alma Mater Studiorum Università di
Bologna/programmazione di reti/Lezioni/Laboratorio 4/codice python Laboratorio 4/
Socket_Programming_Assignment_1.py', wdir='/Users/apirodd/OneDrive - Alma Mater
Studiorum Università di Bologna/programmazione di reti/Lezioni/Laboratorio 4/
codice python Laboratorio 4')
the web server is up on port: 8080
Ready to serve...
```

Cos'è l'indirizzo di Loopback?

L'indirizzo di **loopback** è un indirizzo IP speciale che un computer usa per riferirsi a **sé stesso**. Il valore più comune è **127.0.0.1** per IPv4 e ::1 per IPv6.

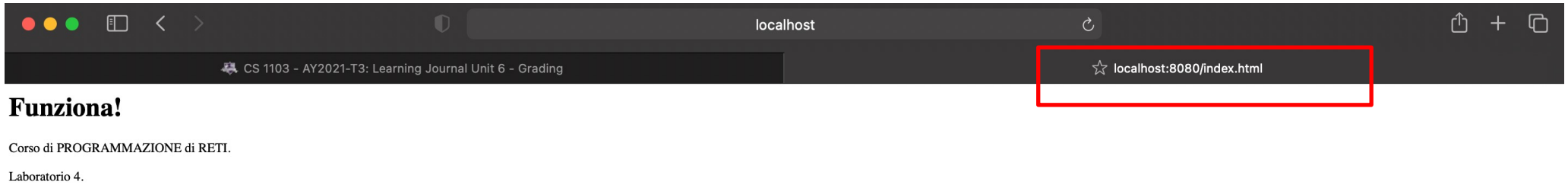
Caratteristica	Loopback (127.0.0.1)	Indirizzo IP di rete
Visibile all'esterno?	❌ No	✅ Sì
Necessita di una scheda di rete?	❌ No	✅ Sì
Usato per debugging?	✅ Sì	❌ No
Supporta servizi locali?	✅ Sì	❌ No

Esercizio 1: Programmazione di un Socket - Web Server

Determinare l'indirizzo IP dell'host che esegue il server (ad es. se è il vostro PC potete considerare come IP «localhost» o 127.0.0.1 indirizzo ip della loopback).

Aprite un browser sulla vostra macchina e digitate nella URL ***http://localhost:8080/index.html***

Dovrebbe aprirsi la pagina sotto



Esercizio 1: Programmazione di un Socket - Web Server

oppure determinate l'IP address associato alla vostra scheda di rete, e da un altro host, aprite un browser e inserite l'URL corrispondente ossia:

http:// «ipaddress-del-vostro-pc»:8080/index.html

«index.html» è il nome del file che avete inserito nella directory del server.

Notate anche l'uso del numero di porta dopo i due punti.

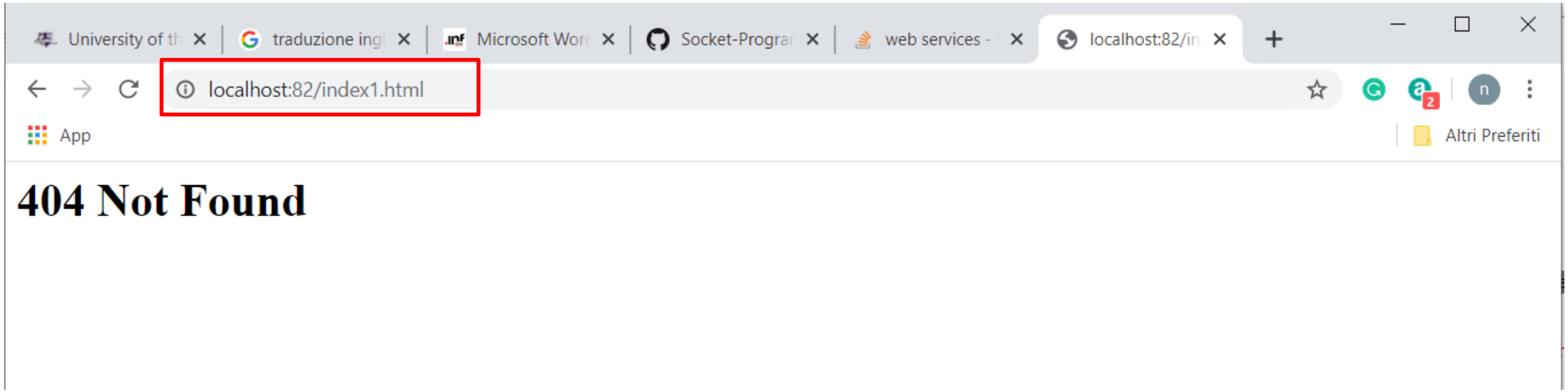
È necessario sostituire questo numero di porta con qualsiasi porta utilizzata nel codice del server. Nell'esempio sopra, abbiamo usato il numero di porta 8080. Il browser dovrebbe quindi visualizzare i contenuti di index.html. Se si omette ": 8080", il browser assumerà la porta 80 e si otterrà la pagina Web dal server solo se il server è in ascolto sulla porta 80.

Esercizio 1: Programmazione di un Socket - Web Server

Quindi provate a ottenere un file che non è presente sul server.

Ossia provate a scrivere nella URL al posto di ***index.html*** il nome ***index1.html***.

Dovreste ricevere un messaggio "***404 Not Found***".



Esercizio 1: Programmazione di un Socket - Web Server

Nel file **TCP_Socket_Server.py** ogni step è opportunamente commentato in modo da spiegare ogni singolo passaggio logico.

Analizzate il file e poi provate a fare il seguente esercizio:

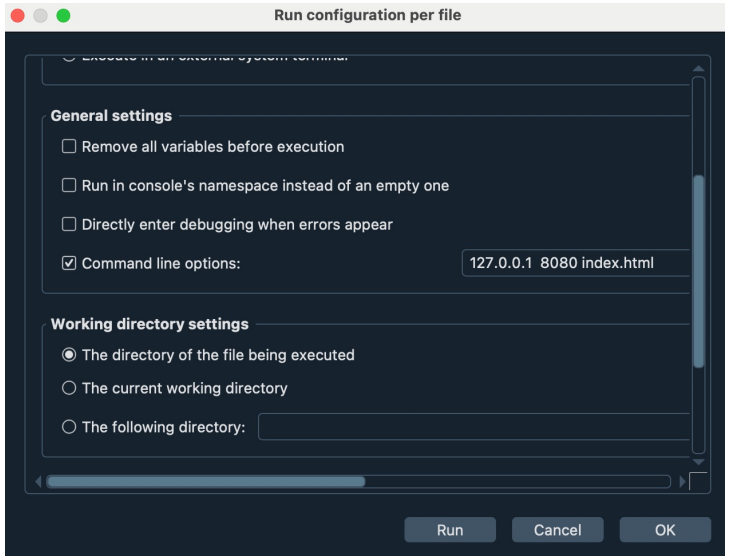
Invece di utilizzare un browser, scrivete il vostro client HTTP per testare il server. Il client si connetterà al server utilizzando una connessione TCP, invierà una richiesta HTTP al server e visualizzerà la risposta del server come output. Si può presumere che la richiesta HTTP inviata sia un metodo GET.

Il client deve accettare gli argomenti della riga di comando specificando l'indirizzo IP o il nome host del server, la porta su cui è in ascolto il server e il percorso in cui l'oggetto richiesto è archiviato sul server. Di seguito è riportato un formato del comando di input per eseguire il client:

```
>>>client.py server_host server_port filename nome_file
```

Cosa dovreste vedere

Lato client



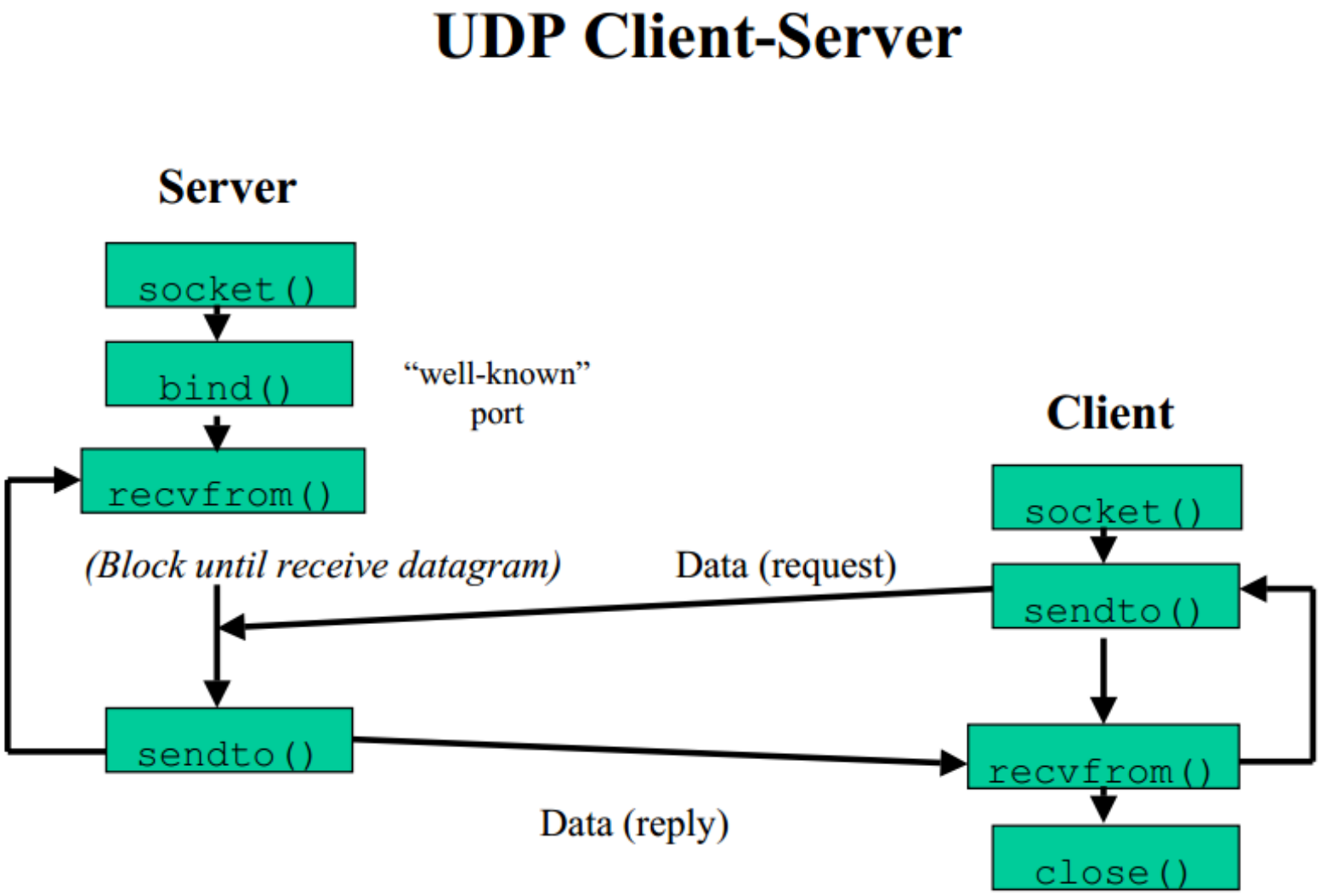

```
In [2]: runfile('/Users/apirodd/Desktop/TCP Socket/TCP_Socket_Client.py', args='127.0.0.1  8080
index.html', wdir='/Users/apirodd/Desktop/TCP Socket')
b'GET /index.html HTTP/1.1 \r\n\r\n'
```

Lato server

```
Desktop/TCP Socket')
the web server is up on port: 8080
Ready to serve...
<socket.socket fd=124, family=AddressFamily.AF_INET, type=SocketKind.SOCK_STREAM, proto=0,
laddr=('127.0.0.1', 8080), raddr=('127.0.0.1', 51922)> ('127.0.0.1', 51922)
b'GET /index.html HTTP/1.1 \r\n\r\n' :: b'GET' : b'/index.html'
b'/index.html' || b'index.html'
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
<html>
<head>
  <meta http-equiv="Content-Type" content="text/html; charset=utf-8">
  <meta http-equiv="Content-Style-Type" content="text/css">
  <title></title>
  <meta name="Generator" content="Cocoa HTML Writer">
  <meta name="CocoaVersion" content="2487.4">
  <style type="text/css">
    p.p2 {margin: 0.0px 0.0px 12.0px 0.0px; font: 12.0px Times; -webkit-text-stroke: #000000}
    span.s1 {font-kerning: none}
  </style>
</head>
```

UDP
SERVER SOCKET

UDP - SERVER SOCKET e CLIENT SOCKET



UDP SERVER SOCKET

UDP_Socket_Server.py - F:\UDP_Socket_Server.py (3.8.0)

File Edit Format Run Options Window Help

```python
'''
                            UDP SERVER SOCKET
Corso di Programmazione di Reti - Laboratorio - Università di Bologna
G.Pau - A. Piroddi
'''

import socket as sk
import time

# Creiamo il socket
sock = sk.socket(sk.AF_INET, sk.SOCK_DGRAM)

# associamo il socket alla porta
server_address = ('localhost', 10000)
print ('\n\r starting up on %s port %s' % server_address)
sock.bind(server_address)

while True:
    print('\n\r waiting to receive message...')
    data, address = sock.recvfrom(4096)

    print('received %s bytes from %s' % (len(data), address))
    print (data.decode('utf8'))


    if data:
        data1='Programmazione di Reti'
        time.sleep(2)
        sent = sock.sendto(data1.encode(), address)
        print ('sent %s bytes back to %s' % (sent, address))
```

Ln: 13 Col: 32

UDP SERVER SOCKET

```
import socket as sk
```

Importiamo il modulo socket nella forma sintatticamente appropriata e lo associamo al nome abbreviato

`sk`

che utilizzeremo nel resto del codice.

UDP SERVER SOCKET

```
import time
```

Importiamo il modulo time.

Il modulo time consente di gestire le attività legate al tempo.

Per esempio se voleste verificare quanto tempo dura l'esecuzione di un codice o di una parte di esso, potete inserire prima dell'inizio del codice l'istruzione

```
t0=time.time()
```

E

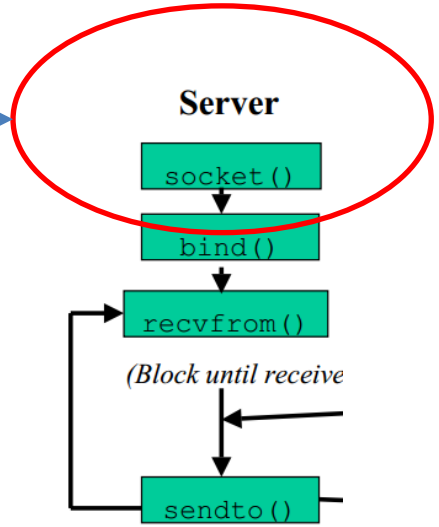
a valle del codice

```
t1=time.time()-t0
Print(t1)
```

in modo da calcolare l'intervallo.

UDP SERVER SOCKET

```
# Creiamo il socket
sock = sk.socket(sk.AF_INET, sk.SOCK_DGRAM)
```



Creiamo il socket UDP e lo associamo alla variabile `sock`

UDP SERVER SOCKET

```
# associamo il socket alla porta
server_address = ('localhost', 10000)
print ('\n\r starting up on %s port %s' % server address)
```

Definiamo l'indirizzo IP del nostro server e la porta su cui lo metteremo in ascolto.

NOTA: se inserite al posto di `localhost` un indirizzo che non è compatibile con la macchina (per esempio 1.2.3.4) vi verrà restituito un errore dal sistema operativo del tipo:

WINDOWS

```
 starting up on 1.2.3.4 port 10000
Traceback (most recent call last):
  File "F:\UDP_Socket_Server.py", line 16, in <module>
    sock.bind(server_address)
OSError: [WinError 10049] Indirizzo richiesto non valido nel proprio contesto
>>>
```

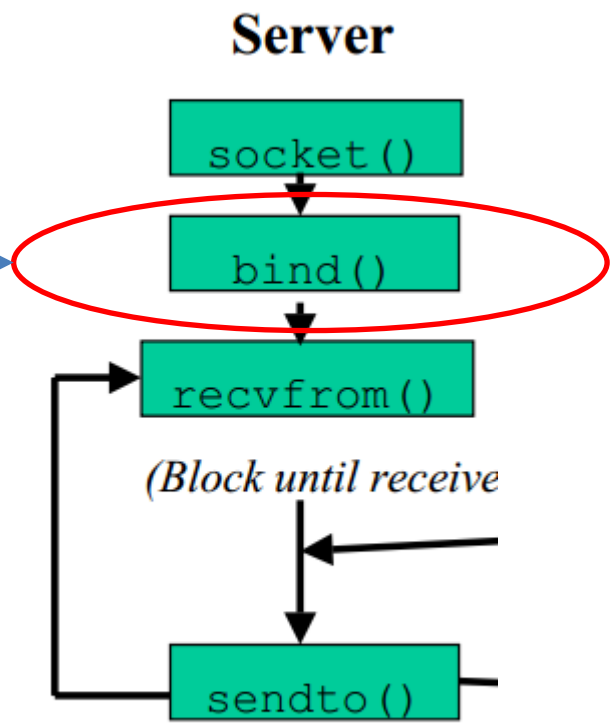
LINUX

```
apirodd@ubuntunet2008:~$ python3 UDP_Socket_Server.py

 starting up on 1.2.3.4 port 10000
Traceback (most recent call last):
  File "UDP_Socket_Server.py", line 16, in <module>
    sock.bind(server_address)
socket.error: [Errno 99] Cannot assign requested address
apirodd@ubuntunet2008:~$
```

UDP SERVER SOCKET

```
sock.bind(server_address)
```

associamo la tupla `server_address` al socket `sock` creato in precedenza

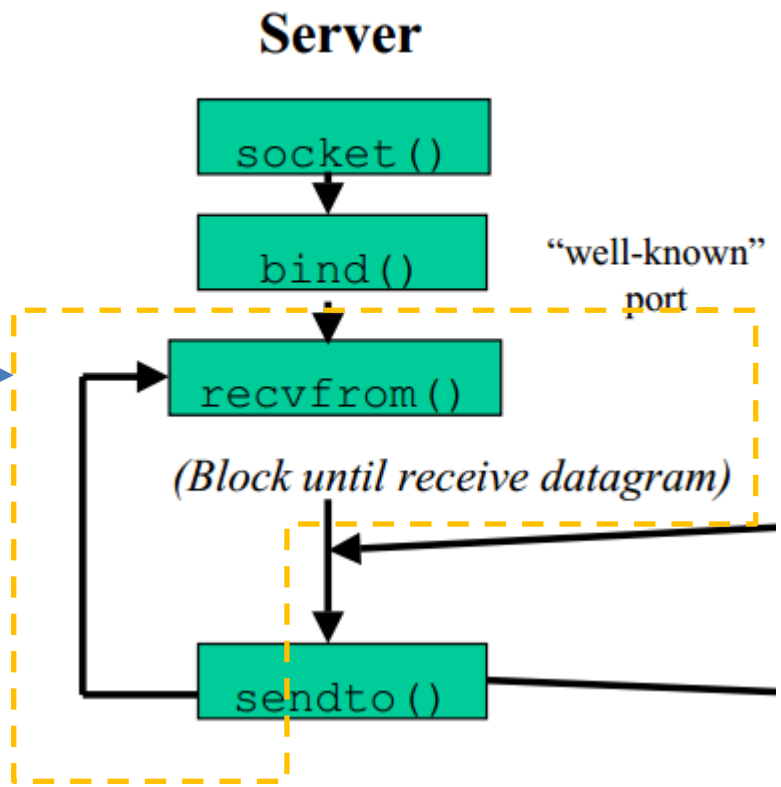


UDP SERVER SOCKET

```
while True:
    print('\n\r waiting to receive message...')
    data, address = sock.recvfrom(4096)
```

Diamo inizio al ciclo while per creare il loop.

E attendiamo l'arrivo di qualche messaggio sul socket `sock` su cui abbiamo impostato la dimensione del buffer a 4096 (*). Quando un messaggio si presenterà ne assegneremo il contenuto alle due variabili `data` e `address`



(*)
socket.recv(bufsize[, flags])
Riceve dati dal socket connesso, restituendo una stringa binaria. **bufsize** specifica la dimensione del buffer di ricezione.
Valori consigliati di bufsize sono potenze di 2 relativamente basse, come 1024, 4096.
flags sono delle flag platform-dependent che modificano il comportamento della funzione.

UDP SERVER SOCKET

```
print('received %s bytes from %s' % (len(data), address))
print (data.decode('utf8'))
```

Visualizziamo quindi la lunghezza dei dati che sono arrivati sul socket e l'indirizzo di provenienza in modo da sapere a chi rispondere.

NOTA:

Python utilizza la formattazione di stringhe in stile C per creare nuove stringhe formattate.

L'operatore "`%`" viene utilizzato per formattare una serie di variabili racchiuse in una "tupla", insieme ad un formato stringa, che contiene testo normale, insieme a "identificatori di argomenti" e simboli speciali come "`% s`" e "`% d`".

Il metodo opposto di `bytes.decode ()` è str.encode (), che restituisce una rappresentazione in byte della stringa Unicode, codificata nella codifica richiesta.

script.py

```
# This prints out "John is 23 years old."
name = "John"
age = 23
print("%s is %d years old." % (name, age))
```

IPython Shell

```
John is 23 years old.

In [1]:
```

```
%s - String (or any object with a string representation, like numbers)

%d - Integers

%f - Floating point numbers

%.<number of digits>f - Floating point numbers with a fixed amount of digits to the right of the dot.

%x/%X - Integers in hex representation (lowercase/uppercase)
```

UDP SERVER SOCKET

```
if data:
    data1='Programmazione di Reti'
    time.sleep(2)
```

Controlla che i dati che sono arrivati sul socket non siano vuoti; in caso la variabile `data` non sia vuota, assegna alla variabile `data1` la stringa

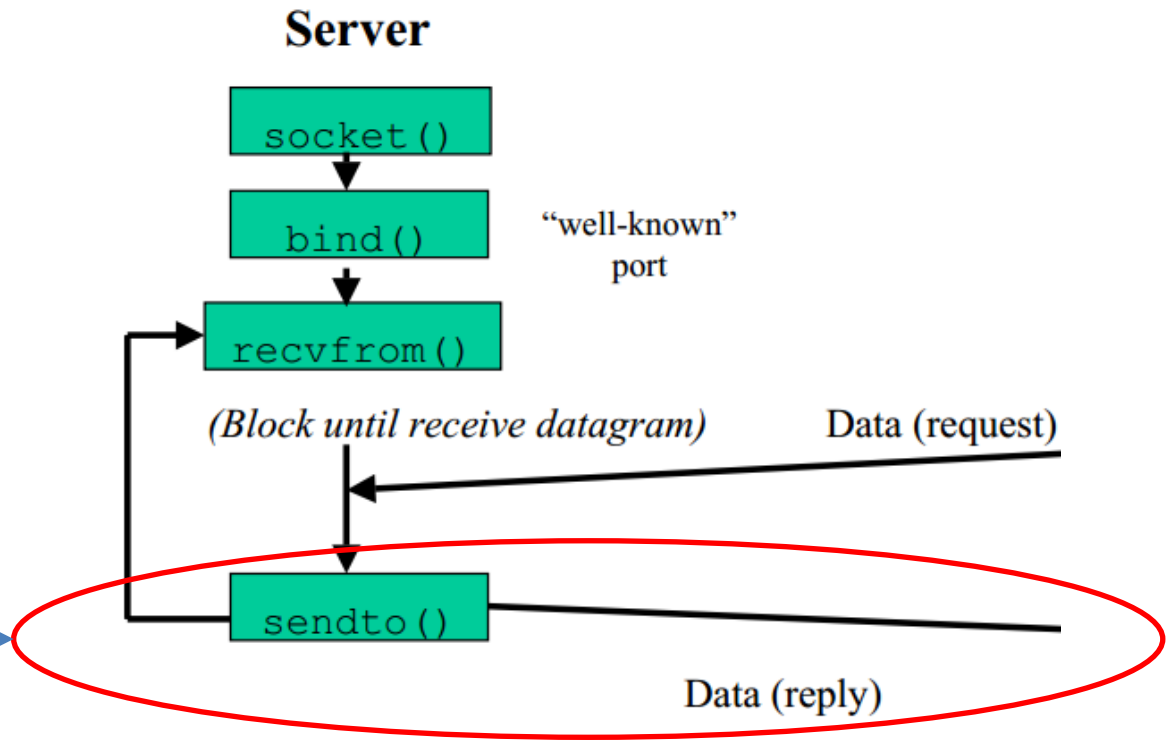
`'Programmazione di Reti'`

e poi attende 2 secondi prima di inviarla verso il client.

UDP SERVER SOCKET

```
sent = sock.sendto(data1.encode(), address)
print ('sent %s bytes back to %s' % (sent, address))
```

Inviamo verso il client (address) il contenuto della variabile data opportunamente codificato.



UDP SERVER SOCKET – Note sull'utilizzo di decode()/encode() in Python3

Principali differenze tra Python 2 e Python 3 riguardo alle stringhe e alla codifica

- **In Python 2, le stringhe (str) erano di tipo byte per default**
  1. In Python 2, il tipo ***str*** rappresentava dati binari (sequenze di byte) e non testo Unicode.
  2. Se volevi lavorare con Unicode, dovevi usare il tipo unicode (u"testo").
  3. La conversione tra ***str*** e ***unicode*** avveniva implicitamente in molti casi, spesso nascondendo problemi di codifica.

- **In Python 3, *str* rappresenta sempre stringhe Unicode**
  1. Il tipo predefinito ***str*** in Python 3 è **nativo Unicode**.
  2. Se hai bisogno di dati binari (sequenze di byte), devi usare bytes (b"testo").
  3. Questo significa che devi essere esplicito nelle conversioni tra ***str*** e ***bytes***, il che può portare a errori di codifica.

Confronto UDP e TCP Server

UDP_Socket_Server.py - F:\UDP_Socket_Server.py (3.8.0)

File Edit Format Run Options Window Help

```
'''
                    UDP SERVER SOCKET
Corso di Programmazione di Reti - Laboratorio - Università di Bologna
G.Pau - A. Piroddi
'''

import socket as sk
import time

# Creiamo il socket
sock = sk.socket(sk.AF_INET, sk.SOCK_DGRAM)

# associamo il socket alla porta
server_address = ('localhost', 10000)
print ('\n\r starting up on %s port %s' % server_address)
sock.bind(server_address)

while True:
    print('\n\r waiting to receive messages...')
    data, address = sock.recvfrom(4096)

    print('received %s bytes from %s' %
    print (data.decode('utf8'))


    if data:
        data1='Programmazione di Reti'
        time.sleep(2)
        sent = sock.sendto(data1.encode(), address)
        print ('sent %s bytes back to %s' % (sent, address))
```

1

4

6

Recv: riceve un messaggio su di un certo socket

Recvfrom: riceve un messaggio su di un certo socket (non orientato alla connessione)

Send: invia un messaggio attraverso un socket

Sendto: invia un messaggio attraverso un socket (non orientato alla connessione)

Ln: 28 Col: 0

TCP_Socket_Server.py - F:/TCP_Socket_Server.py (3.8.0)

File Edit Format Run Options Window Help

```
# Corso di Programmazione di Reti - Laboratorio - Università di Bologna
# Socket_Programming_Assignment - WebServer - G.Pau - A. Piroddi

import sys
from socket import *

serverSocket = socket(AF_INET, SOCK_STREAM)
server_address=('localhost',8080)
serverSocket.bind(server_address)

#listen(1) Definisce la lunghezza della coda di backlog, ovvero il numero
#di connessioni in entrata che sono state completate dallo stack TCP / IP
#ma non ancora accettate dall'applicazione.
serverSocket.listen(1)
print ('the web server is up on port:',8080)

while True:

    print ('Ready to serve...')
    connectionSocket, addr = serverSocket.accept()
    print(connectionSocket,addr)

                                  socket.recv(1024)
                                  ())>0:
            print (message,'::',message.split()[0],':',message.split()[1])
            filename = message.split()[1]
            print (filename,'||',filename[1:])
            f = open(filename[1:],'r+')
            outputdata = f.read()
            print (outputdata)

    #Invia la riga di intestazione HTTP nel socket con il messaggio OK

            connectionSocket.send("HTTP/1.1 200 OK\r\n\r\n".encode())
            connectionSocket.send(outputdata.encode())
            connectionSocket.send("\r\n".encode())
            connectionSocket.close()


    except IOError:
#Invia messaggio di risposta per file non trovato
        connectionSocket.send(bytes("HTTP/1.1 404 Not Found\r\n\r\n","UTF-8"))
        connectionSocket.send(bytes("<html><head></head><body><h1>404 Not Found<
        connectionSocket.close()
```

1

2

3

4

5

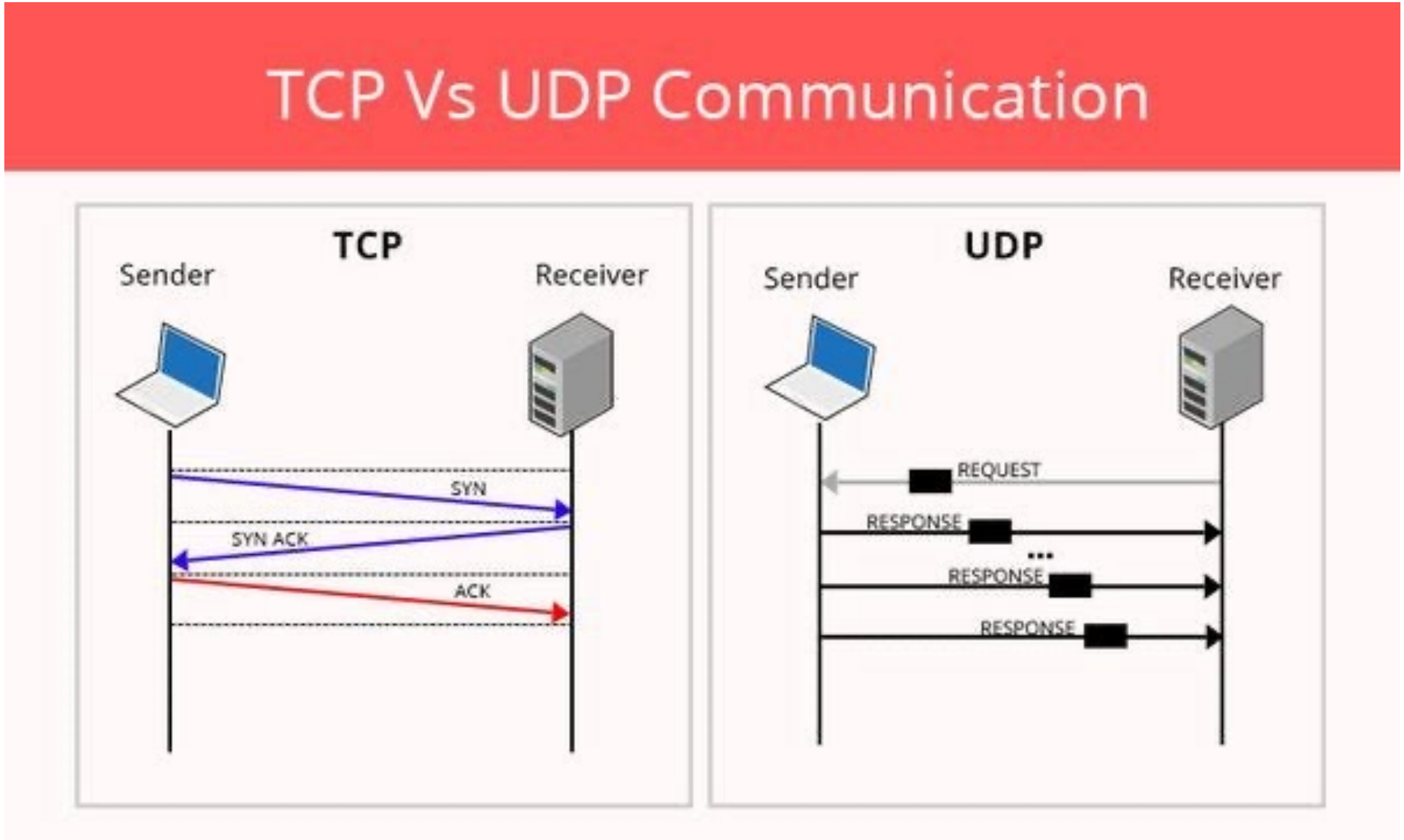
6

7

Ln: 15 Col: 0

UDP
CLIENT SOCKET

UDP CLIENT SOCKET



UDP CLIENT SOCKET

UDP_Socket_Client.py - F:\UDP_Socket_Client.py (3.8.0)

File Edit Format Run Options Window Help

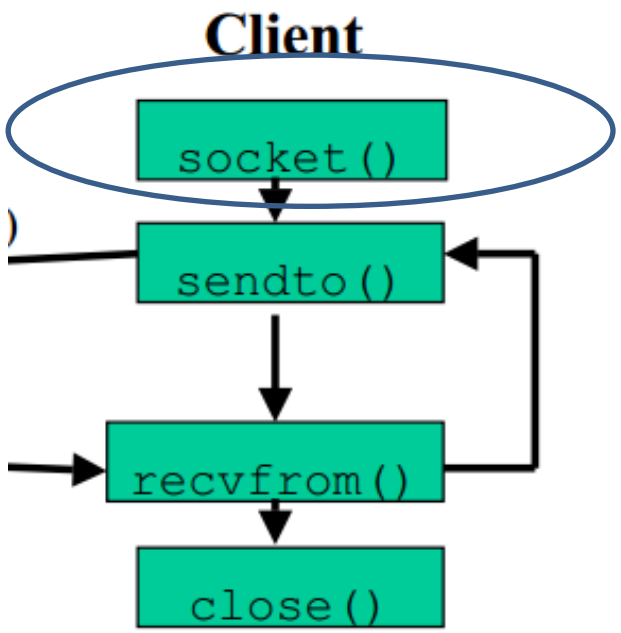
```python
'''
                        UDP CLIENT SOCKET
Corso di Programmazione di Reti - Laboratorio - Università di Bologna
G.Pau - A. Piroddi
'''

import socket as sk
import time


# Create il socket UDP
sock = sk.socket(sk.AF_INET, sk.SOCK_DGRAM)

server_address = ('localhost', 10000)
message = 'Questo è il corso di ?'

try:

    # inviate il messaggio
    print ('sending "%s"' % message)
    time.sleep(2) #attende 2 secondi prima di inviare la richiesta
    sent = sock.sendto(message.encode(), server_address)

    # Ricevete la risposta dal server
    print('waiting to receive')
    data, server = sock.recvfrom(4096)
    time.sleep(2)
    print ('received message "%s"' % data.decode('utf8'))
except Exception as info:
    print(info)
finally:
    print ('closing socket')
    sock.close()
```

Ln: 1 Col: 0

UDP CLIENT SOCKET

```
# Create il socket UDP
sock = sk.socket(sk.AF_INET, sk.SOCK_DGRAM)
```

Creiamo il socket UDP per l'invio della richiesta e lo associamo alla variabile `sock`



UDP CLIENT SOCKET

Associamo alla variabile *server_address* la tupla che contiene l'indirizzo e la porta **udp** del server.

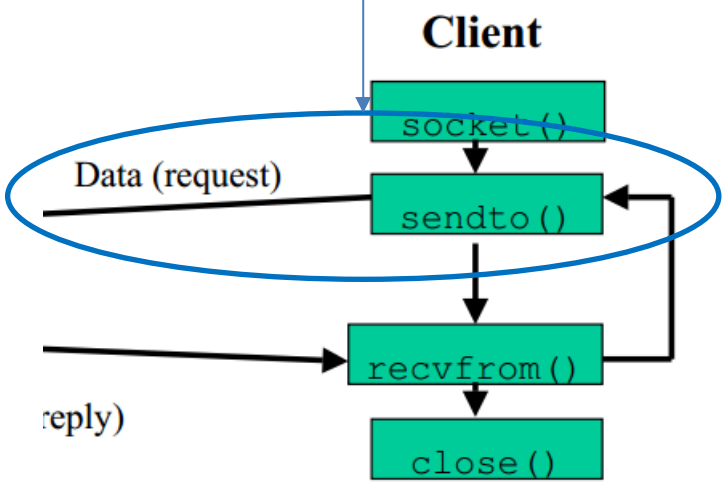
```
server_address = ('localhost', 10000)
message = 'Questo è il corso di ?'
```

Prepariamo la variabile *message* assegnandole la stringa che vogliamo inviare al server

UDP CLIENT SOCKET

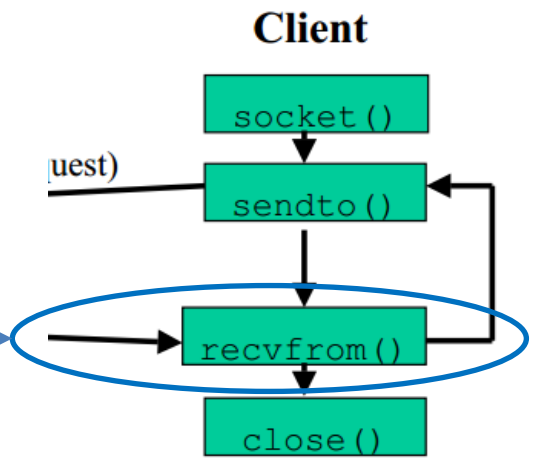
```
sent = sock.sendto(message.encode(), server_address)
```

Inviamo, tramite il modulo socket (*sock.sendto()*, il messaggio all'indirizzo del server (ipaddress e udp port) e associamo il contenuto di alla variabile *sent*



UDP CLIENT SOCKET

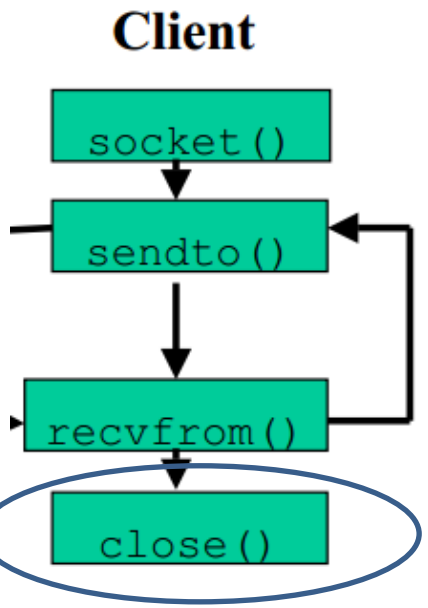
```
data, server = sock.recvfrom(4096)
```



Appena riceviamo il contenuto del messaggio proveniente dal server tramite il modulo socket *sock.recvfrom()* ne assegniamo il contenuto alle due variabili *data* e *server*, corrispondenti rispettivamente al PAYLOAD e all'indirizzo del server (ip address e porta) da cui esso proviene.

UDP CLIENT SOCKET

```
sock.close()
```



Al termine della ricezione del datagram da parte del server, chiudiamo il socket.

Confronto UDP e TCP Client

UDP_Socket_Client.py - F:\UDP_Socket_Client.py (3.8.0)

File Edit Format Run Options Window Help

```python
'''
                        UDP CLIENT SOCKET
Corso di Programmazione di Reti - Laboratorio - Università di Bologna
G.Pau - A. Piroddi
'''

import socket as sk
import time

# Create il socket UDP
sock = sk.socket(sk.AF_INET, sk.SOCK_DGRAM)

server_address = ('localhost', 10000)
message = 'Questo è il corso di ?'

try:

    # inviate il messaggio
    print ('sending "%s"' % message)
    time.sleep(2) #attende 2 secondi prima di inviare la richiesta
    sent = sock.sendto(message.encode(), server_address)

    # Ricevete la risposta dal server
    print('waiting to receive from')
    data, server = sock.recvfrom(4096)
    #print(server)
    time.sleep(2)
    print ('received message "%s"' % data.decode('utf8'))
except Exception as info:
    print(info)
finally:
    print ('closing socket')
    sock.close()
```

Ln: 36 Col: 0

TCP_Socket_Client.py - F:/TCP_Socket_Client.py (3.8.0)

File Edit Format Run Options Window Help

```python
'''
Corso di Programmazione di Reti - Laboratorio - Università di Bologna
Socket_Programming_Assignment - WebServer - G.Pau - A. Piroddi

Per eseguire il presente codice è necessario utilizzare o una Command Prompt o d

'''
import socket as sk
import sys

clientsocket = sk.socket(sk.AF_INET, sk.SOCK_STREAM)

if len(sys.argv) != 4:
    print (len(sys.argv))
    print("Il comando non è corretto. Usa il seguente formato: client.py indiriz
    sys.exit(0)

host = str(sys.argv[1])
port = int(sys.argv[2])
request = str(sys.argv[3])
request = "GET /" + request + " HTTP/1.1"
try:
    clientsocket.connect((host,port))
except Exception as data:
    print (Exception,":",data)
    print ("Ritenta sarai più fortunato.\r\n")
    sys.exit(0)
clientsocket.send(request.encode()) #se tutto è andato bene, rispondiamo al serv
print(request.encode())
response = clientsocket.recv(1024)

print (response)

clientsocket.close()
```

Ln: 21 Col: 0

WIRESHARK

ANALIZZATORE di PROTOCOLLO - WIRESHARK

Ai link seguenti potete trovare il manuale di Wireshark e i pacchetti di installazione.

Manuale Wireshark:
https://www.wireshark.org/download/docs/user-guide.pdf

Download Pacchetto installativo:
https://www.wireshark.org/download.html

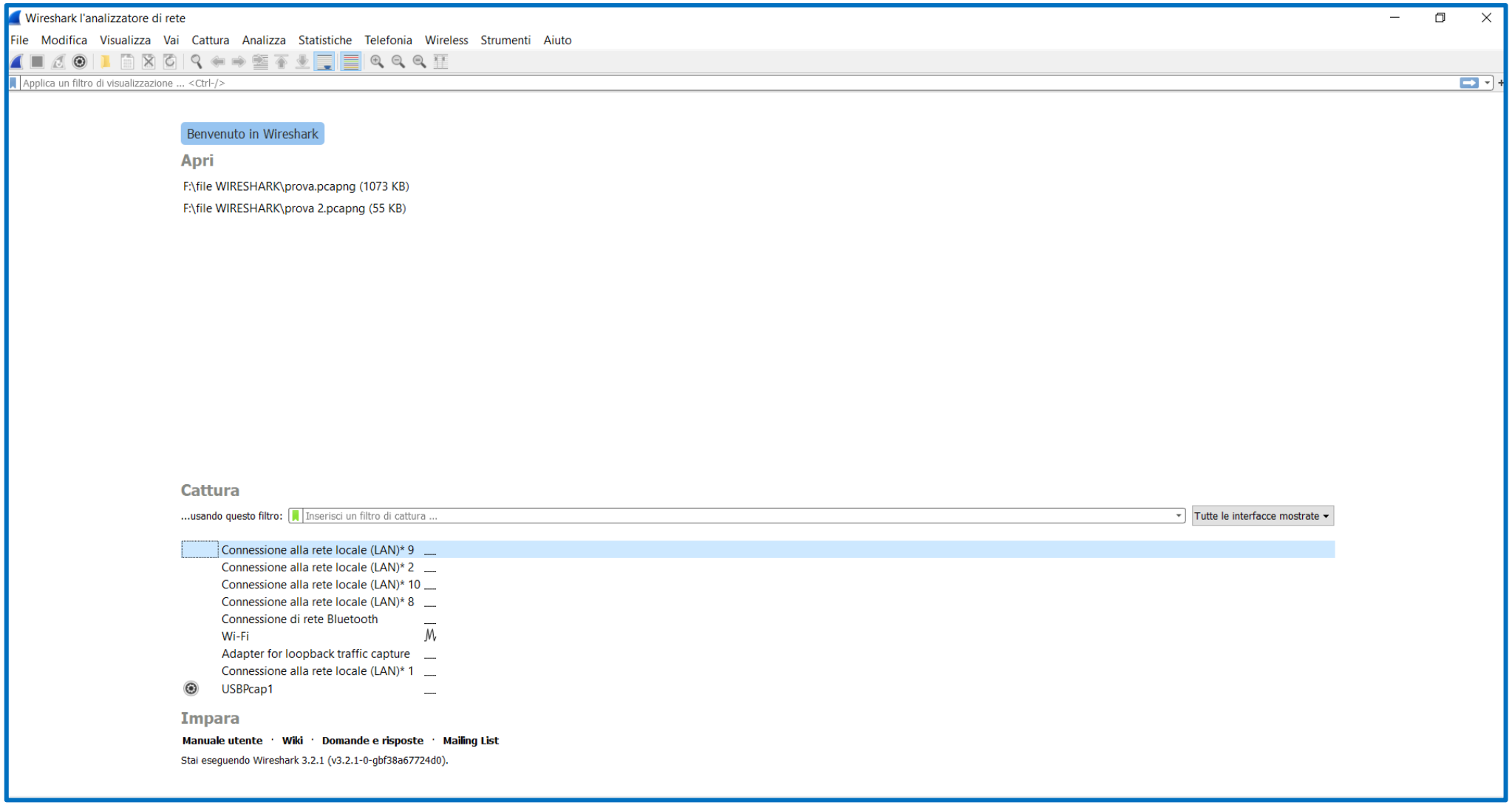
Stable Release (3.2.2)

- Windows Installer (64-bit)
- Windows Installer (32-bit)
- Windows PortableApps® (32-bit)
- macOS Intel 64-bit .dmg
- Source Code

VENDOR / PLATFORM	SOURCES
Alpine / Alpine Linux	Standard package
Apple / macOS	Homebrew (Formula) MacPorts Fink
Arch Linux / Arch Linux	Standard package
Canonical / Ubuntu	Standard package Latest stable PPA
Debian / Debian GNU/Linux	Standard package
The FreeBSD Project / FreeBSD	Standard package
Gentoo Foundation / Gentoo Linux	Standard package
HP / HP-UX	Porting And Archive Centre for HP-UX
NetBSD Foundation / NetBSD	Standard package
Novell / openSUSE, SUSE Linux	Standard package
Offensive Security / Kali Linux	Standard package
PCLinuxOS / PCLinuxOS	Standard package
Red Hat / Fedora	Standard package
Red Hat / Red Hat Enterprise Linux	Standard package
Slackware Linux / Slackware	SlackBuilds.org
Oracle / Solaris 11	Standard package
* / *	The Written Word

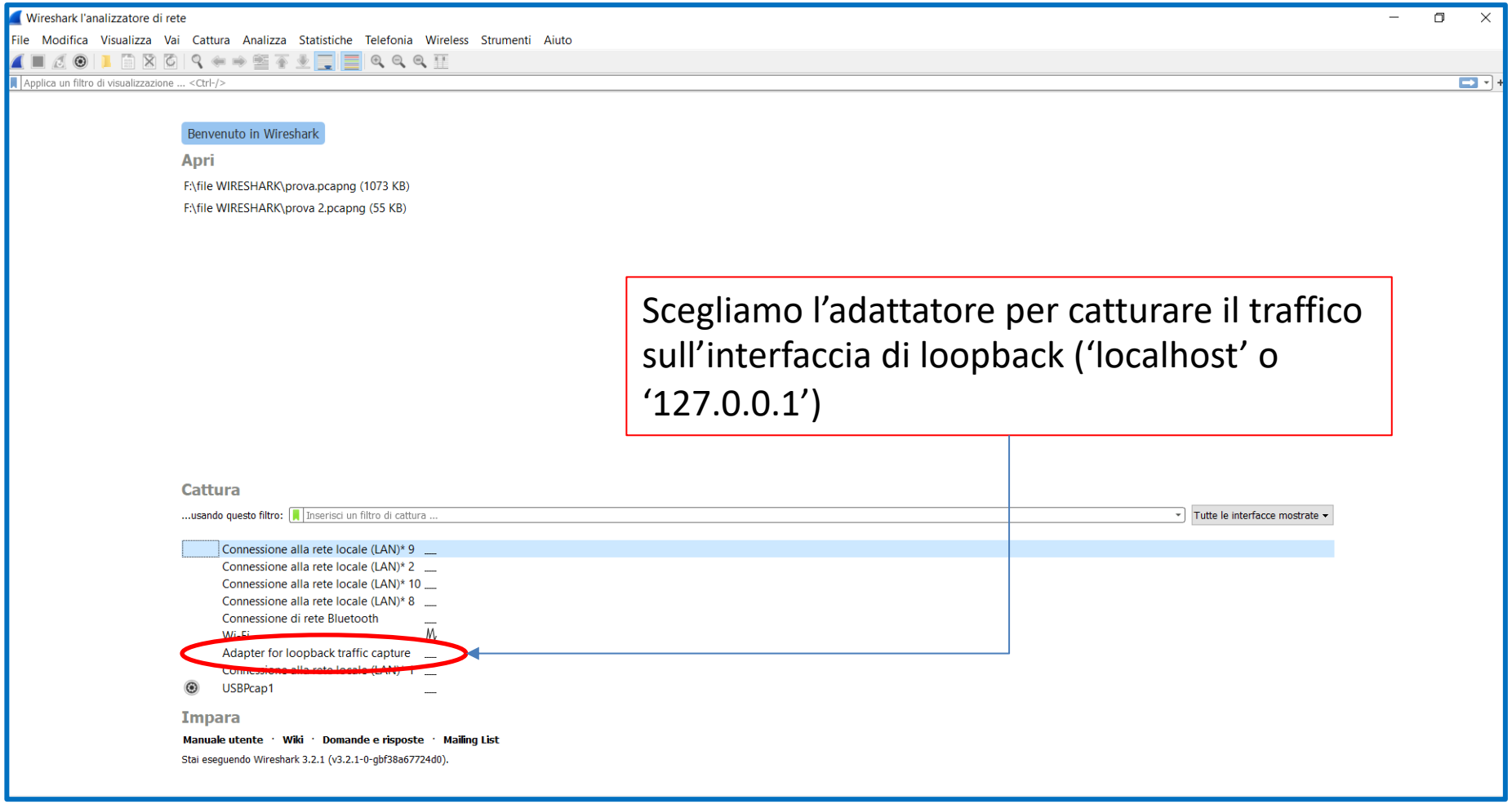
ANALIZZATORE di PROTOCOLLO - WIRESHARK

L’interfaccia grafica si presenta cosi:



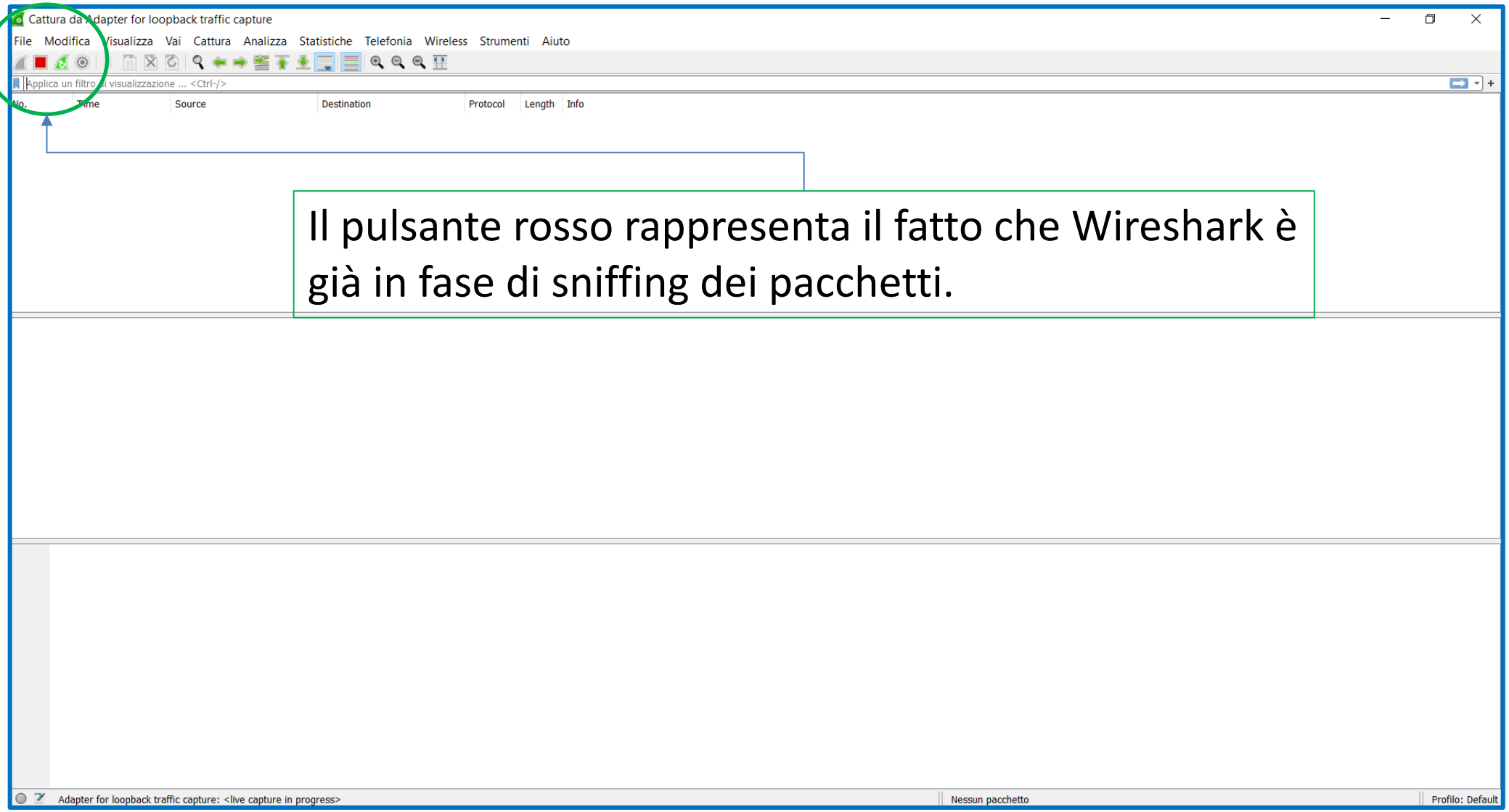
ANALIZZATORE di PROTOCOLLO - WIRESHARK

Selezionare l'interfaccia di rete su cui si vuole effettuare lo sniffing dei pacchetti



Scegliamo l'adattatore per catturare il traffico sull'interfaccia di loopback ('localhost' o '127.0.0.1')

ANALIZZATORE di PROTOCOLLO - WIRESHARK



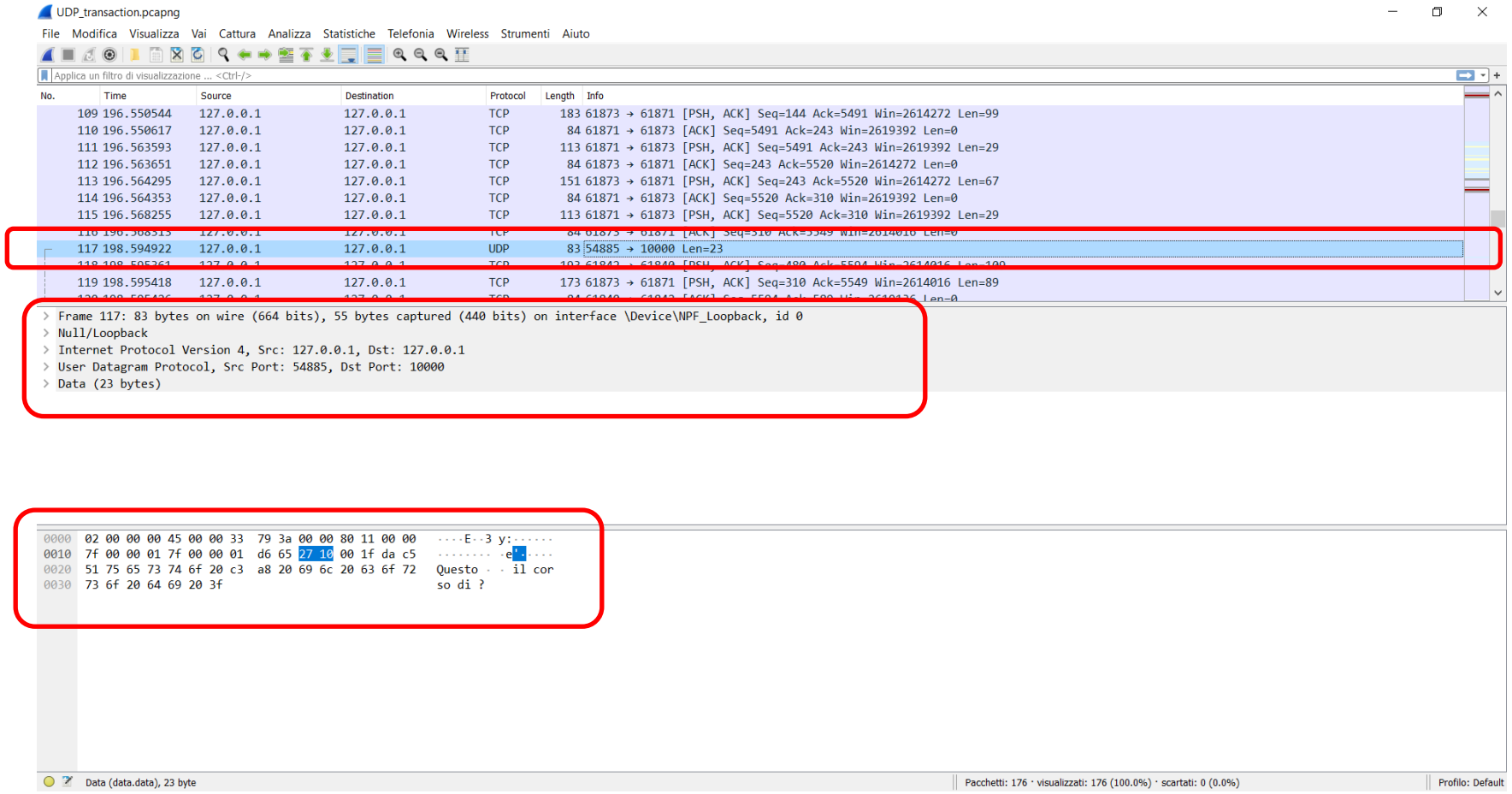
Il pulsante rosso rappresenta il fatto che Wireshark è già in fase di sniffing dei pacchetti.

ANALIZZATORE di PROTOCOLLO - WIRESHARK

Wireshark è attivo e sta sniffando il traffico che proviene ed è destinato alla interfaccia di loopback.

Lanciate quindi l'UDP_Socket_Server.py e successivamente l'UDP_Socket_Client.py

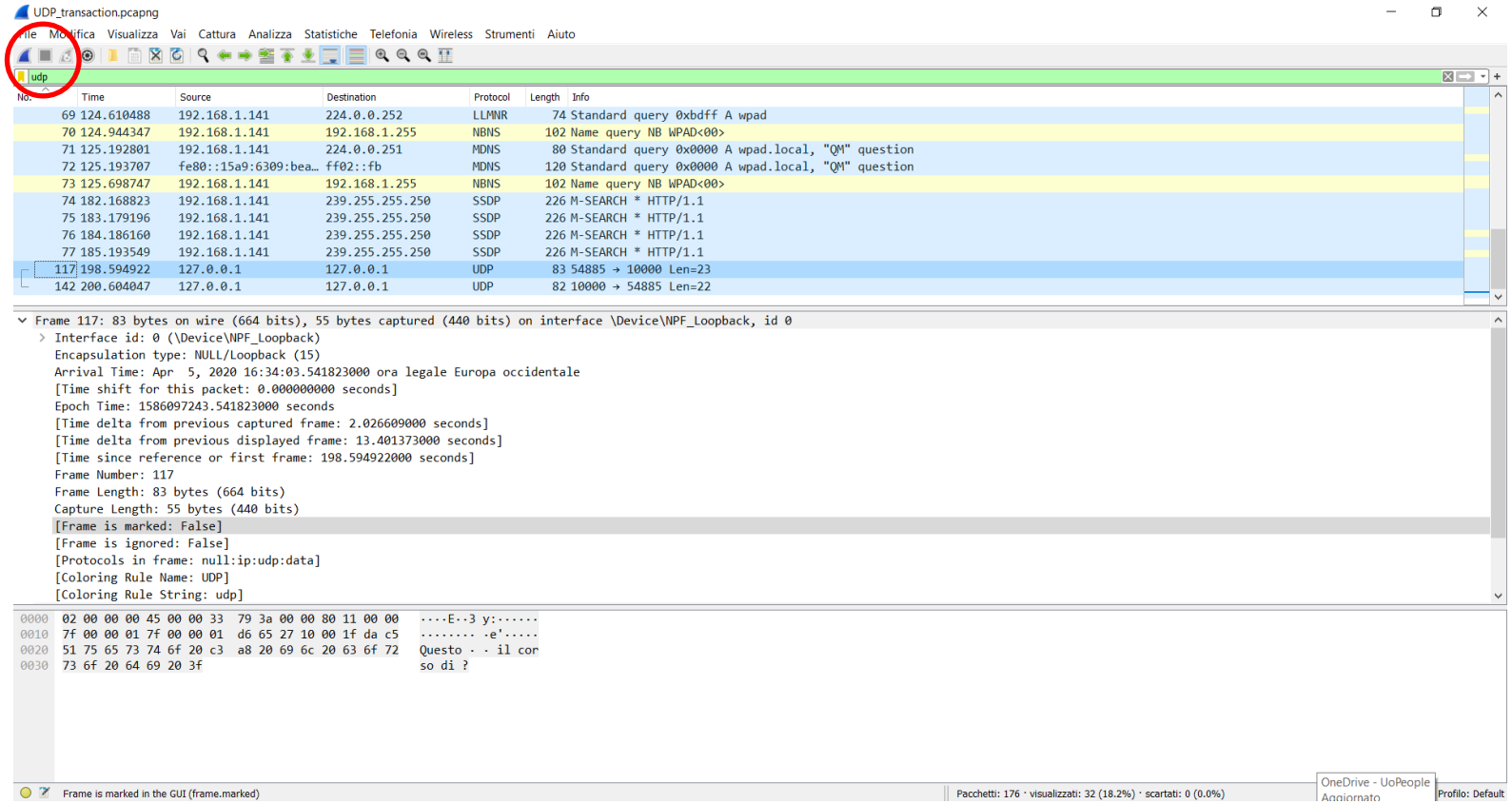
Comincerete a vedere nell'interfaccia di Wireshark alcune righe informative del tipo:



ANALIZZATORE di PROTOCOLLO - WIRESHARK

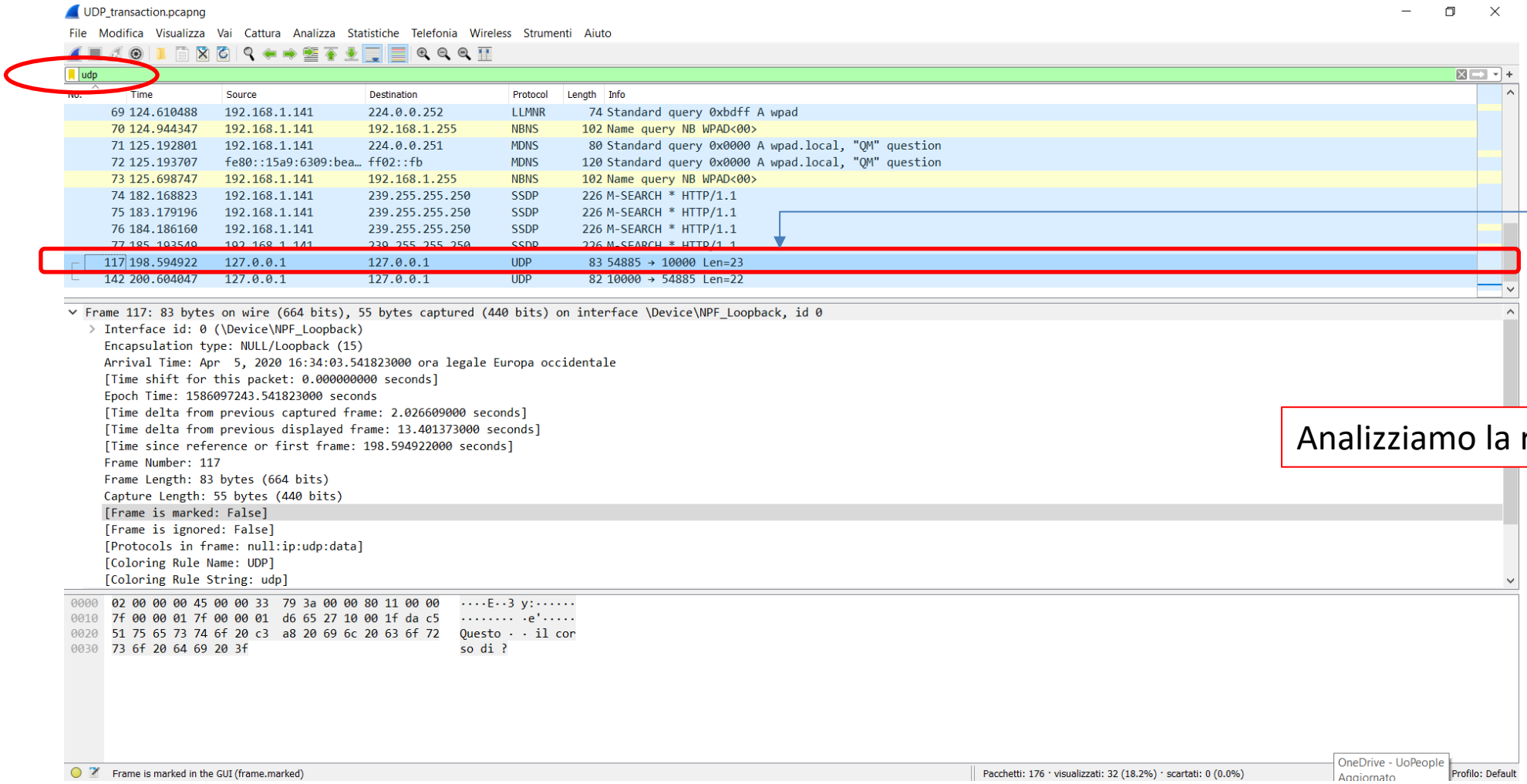
Stoppiamo lo sniffing,

E salviamo il file in modo da averlo disponibile per successive analisi.



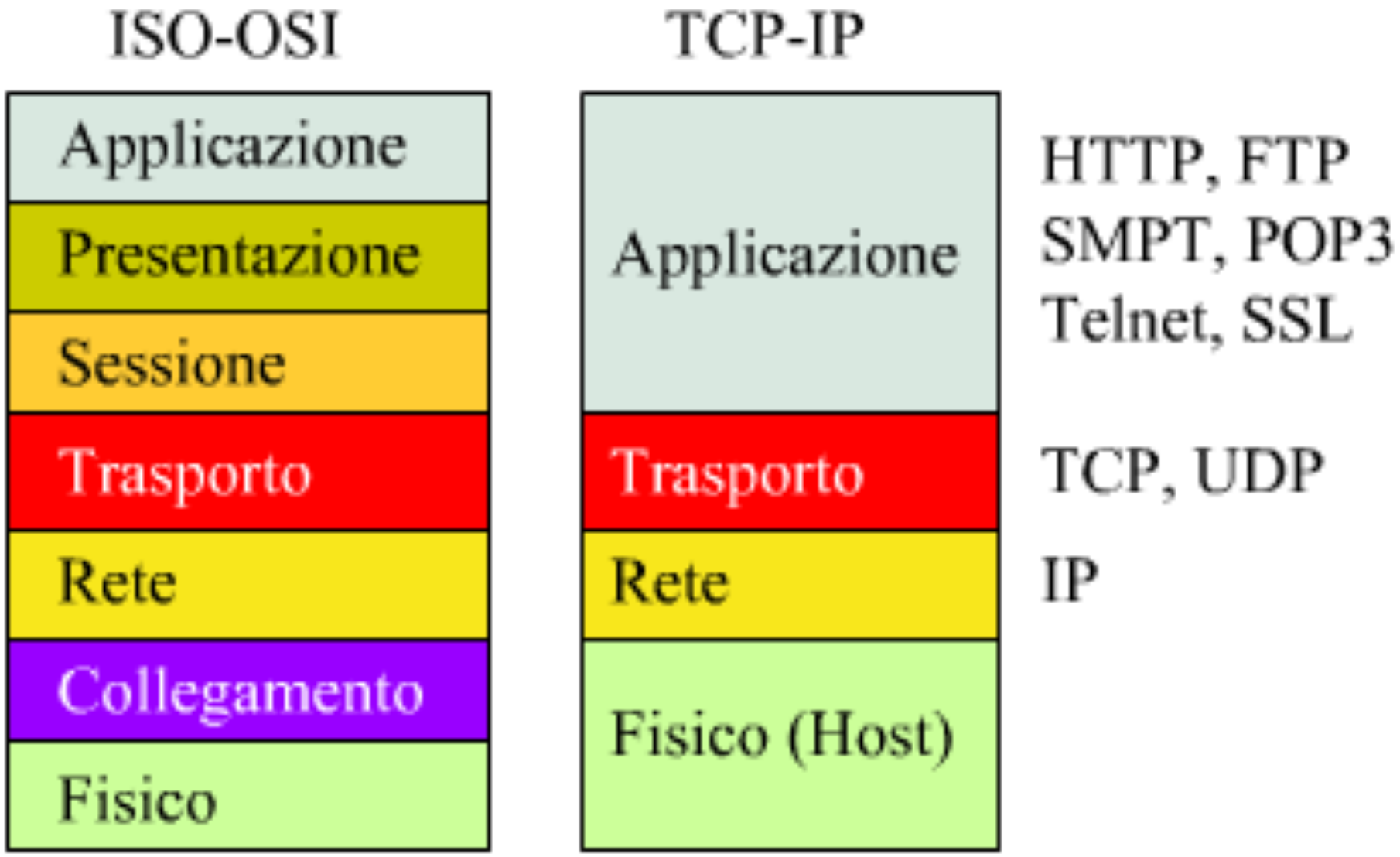
ANALIZZATORE di PROTOCOLLO - WIRESHARK

Visualizziamo il solo traffico UDP.
NOTA: stiamo solo filtrando la visualizzazione non stiamo filtrando il traffico catturato, cosa che invece è possibile fare utilizzando i filtri di cattura.



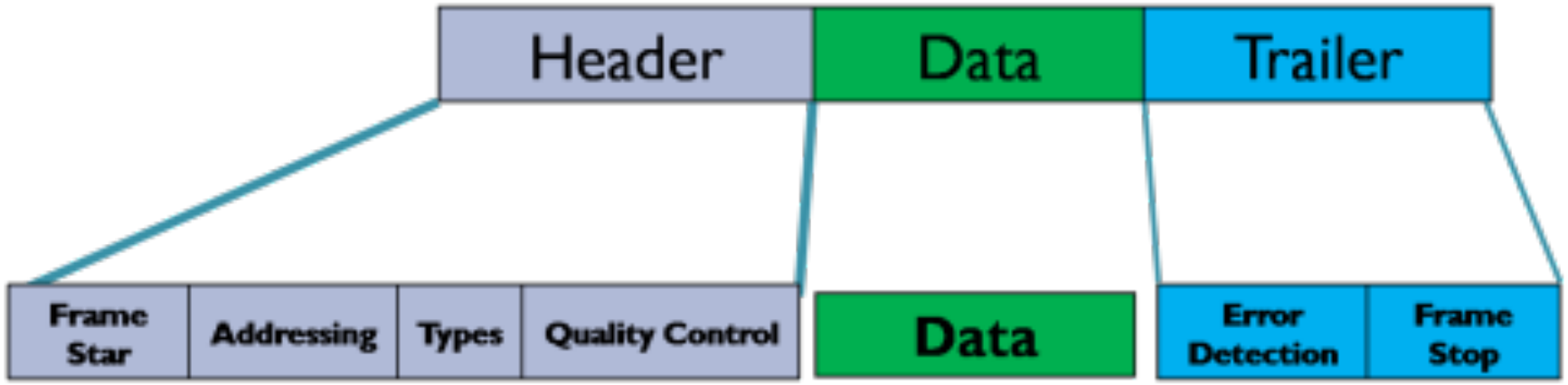
Analizziamo la richiesta del client

Pila TCP/IP


ISO-OSI
TCP-IP
Applicazione
Presentazione
Sessione
Trasporto
Rete
Collegamento
Fisico
Applicazione
Trasporto
Rete
Fisico (Host)
HTTP, FTP
SMPT, POP3
Telnet, SSL
TCP, UDP
IP

ANALIZZATORE di PROTOCOLLO – WIRESHARK - FRAME

Il ***livello data link*** (Collegamento) si occupa di fornire ai livelli superiori una linea di comunicazione esente da errori di trasmissione non segnalati; per fare questo decompone i dati del mittente in pacchetti chiamati frame, composti da alcune centinaia o migliaia di byte, e li spedisce in sequenza attendendo eventualmente la conferma di avvenuta ricezione da parte del destinatario.



Analisi di una cattura TCP - *Struttura di un pacchetto TCP*

Campo	Significato
Source Port	Porta di origine (es. 55000, un numero casuale del client)
Destination Port	Porta di destinazione (es. 80 per HTTP)
Sequence Number	Numero progressivo del segmento TCP
Acknowledgment Number	Conferma della ricezione dei dati
Flags	Controllo della connessione (SYN, ACK, FIN, etc.)
Window Size	Controlla il flusso dei dati
Checksum	Verifica l'integrità del pacchetto

ANALIZZATORE di PROTOCOLLO – WIRESHARK - FRAME

Esempio pratico: Connessione TCP (Three-Way Handshake)

1. **SYN** → Il client invia un pacchetto SYN per iniziare la connessione.
2. **SYN-ACK** → Il server risponde con un SYN-ACK.
3. **ACK** → Il client conferma con un ACK e la connessione è stabilita.

Come individuarlo in Wireshark?

- Filtra con ***tcp.flags.syn == 1*** per vedere i pacchetti SYN.
- Filtra con ***tcp.flags.ack == 1*** per vedere le conferme.

Analisi di una cattura UDP - *Struttura di un pacchetto UDP*

Campo	Significato
Source Port	Porta del mittente
Destination Port	Porta del destinatario (es. 53 per DNS)
Length	Lunghezza del pacchetto
Checksum	Controlla l'integrità dei dati

ANALIZZATORE di PROTOCOLLO – WIRESHARK - FRAME

Esempio pratico: Query DNS con UDP

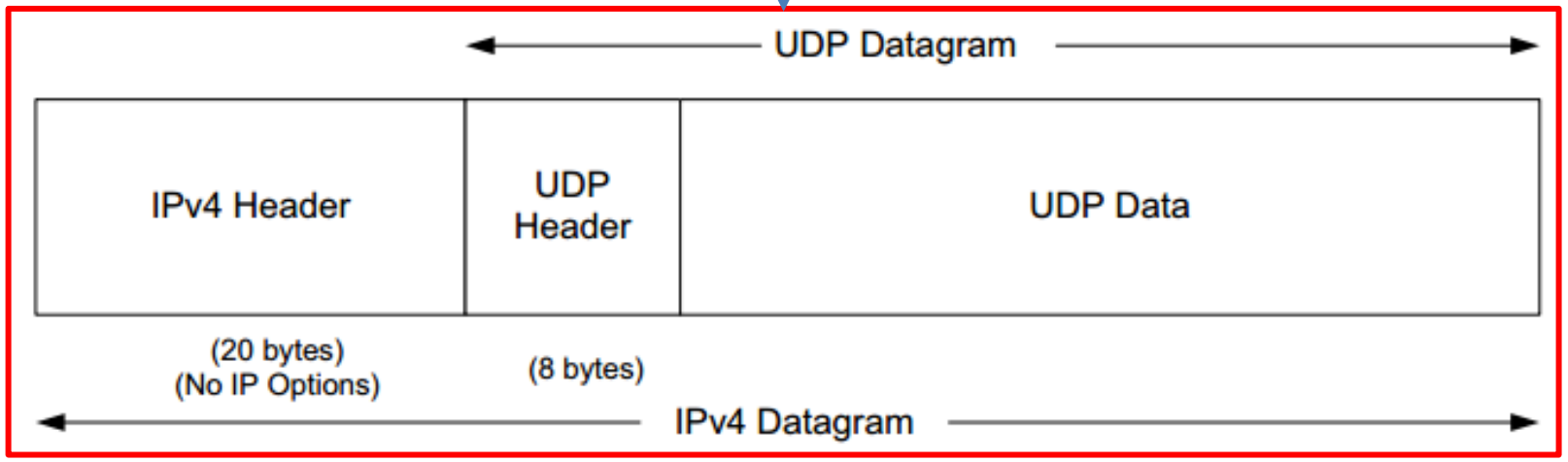
1. Un client invia una richiesta DNS a un server (udp.port == 53).
2. Il server risponde con l'indirizzo IP richiesto.

Come individuare il traffico DNS in Wireshark?

- Usa il filtro ***udp.port == 53*** per vedere solo pacchetti DNS.
- Guarda la sezione **"Queries"** per le richieste e **"Responses"** per le risposte.

ANALIZZATORE di PROTOCOLLO – WIRESHARK - TRASPORTO

```
User Datagram Protocol, Src Port: 54885, Dst Port: 10000
    Source Port: 54885
    Destination Port: 10000
    Length: 31
    Checksum: 0xdac5 [unverified]
    [Checksum Status: Unverified]
    [Stream index: 9]
  > [Timestamps]
```

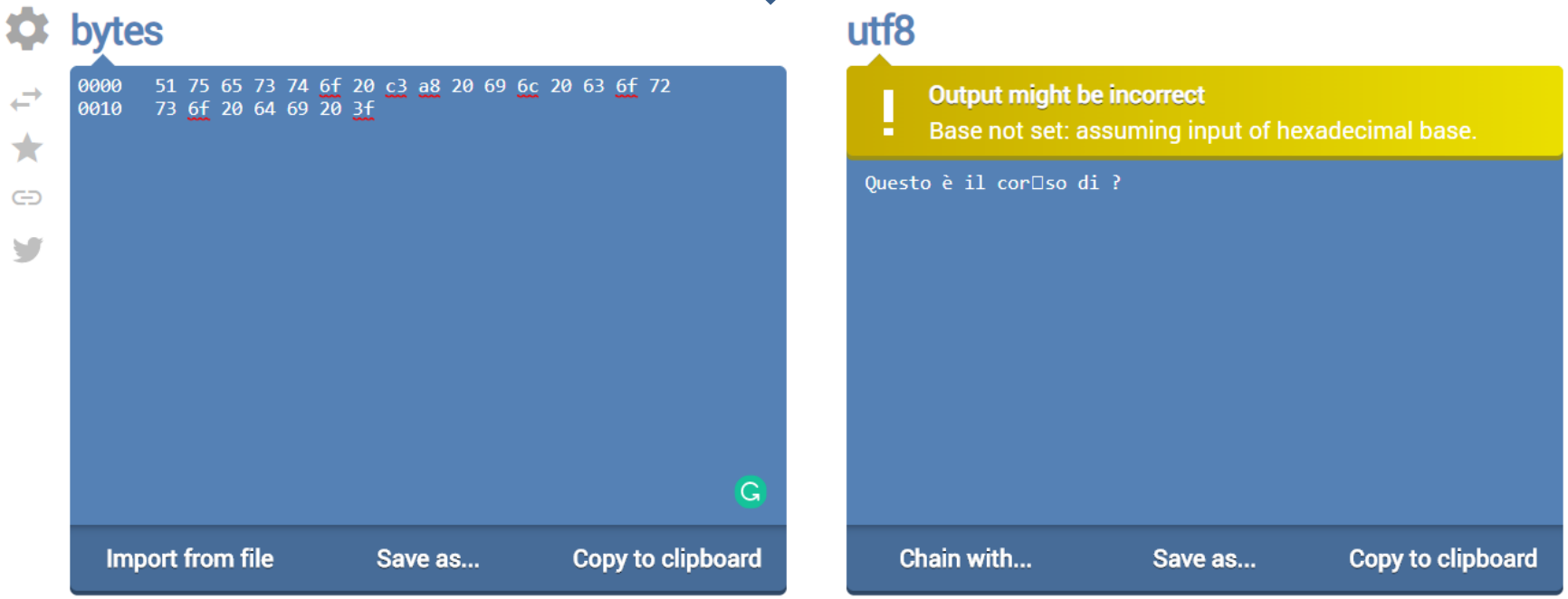


ANALIZZATORE di PROTOCOLLO – WIRESHARK - DATA

```
Data (23 bytes)
    Data: 51756573746f20c3a820696c20636f72736f206469203f
    [Length: 23]

0000  02 00 00 00 45 00 00 33  79 3a 00 00 80 11 00 00   ····E··3 y:******
0010  7f 00 00 01 7f 00 00 01  d6 65 27 10 00 1f da c5   ********  ·e******
0020  51 75 65 73 74 6f 20 c3  a8 20 69 6c 20 63 6f 72   Questo ·· il cor
0030  73 6f 20 64 69 20 3f                               so di ?
```

Se provate ad accedere al seguente link

https://onlineutf8tools.com/convert-bytes-to-utf8



ANALIZZATORE di PROTOCOLLO - WIRESHARK

UDP_transaction.pcapng

File Modifica Visualizza Vai Cattura Analizza Statistiche Telefonia Wireless Strumenti Aiuto

udp

No.	Time	Source	Destination	Protocol	Length	Info
69	124.610488	192.168.1.141	224.0.0.252	LLMNR	74	Standard query 0xbdff A wpad
70	124.944347	192.168.1.141	192.168.1.255	NBNS	102	Name query NB WPAD<00>
71	125.192801	192.168.1.141	224.0.0.251	MDNS	80	Standard query 0x0000 A wpad.local, "QM" question
72	125.193707	fe80::15a9:6309:bea…	ff02::fb	MDNS	120	Standard query 0x0000 A wpad.local, "QM" question
73	125.698747	192.168.1.141	192.168.1.255	NBNS	102	Name query NB WPAD<00>
74	182.168823	192.168.1.141	239.255.255.250	SSDP	226	M-SEARCH * HTTP/1.1
75	183.179196	192.168.1.141	239.255.255.250	SSDP	226	M-SEARCH * HTTP/1.1
76	184.186160	192.168.1.141	239.255.255.250	SSDP	226	M-SEARCH * HTTP/1.1
77	185.193549	192.168.1.141	239.255.255.250	SSDP	226	M-SEARCH * HTTP/1.1
117	198.594922	127.0.0.1	127.0.0.1	UDP	83	54885 → 10000 Len=23
142	200.604047	127.0.0.1	127.0.0.1	UDP	82	10000 → 54885 Len=22

```
> Frame 142: 82 bytes on wire (656 bits), 54 bytes captured (432 bits) on interface \Device\NPF_Loopback, id 0
v Null/Loopback
    Family: IP (2)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
> User Datagram Protocol, Src Port: 10000, Dst Port: 54885
> Data (22 bytes)
```

Analizziamo la risposta del server

```
0000  02 00 00 00 45 00 00 32  79 53 00 00 80 11 00 00   ····E··2 yS======
0010  7f 00 00 01 7f 00 00 01  27 10 d6 65 00 1e da ee   ======== '··e····
0020  50 72 6f 67 72 61 6d 6d  61 7a 69 6f 6e 65 20 64   Programm azione d
0030  69 20 52 65 74 69                                  i Reti
```

Data (data.data), 22 byte | Pacchetti: 176 · visualizzati: 32 (18.2%) · scartati: 0 (0.0%) | Profilo: Default

ANALIZZATORE di PROTOCOLLO - WIRESHARK

```
142 200.604047    127.0.0.1    127.0.0.1    UDP    82 10000 → 54885 Len=22
```

```
> Frame 142: 82 bytes on wire (656 bits), 54 bytes captured (432 bits) on interface \Device\NPF_Loopback, id 0
v Null/Loopback
    Family: IP (2)
> Internet Protocol Version 4, Src: 127.0.0.1, Dst: 127.0.0.1
v User Datagram Protocol, Src Port: 10000, Dst Port: 54885
    Source Port: 10000
    Destination Port: 54885
    Length: 30
    Checksum: 0xdaee [unverified]
    [Checksum Status: Unverified]
    [Stream index: 9]
  > [Timestamps]
v Data (22 bytes)
    Data: 50726f6772616d6d617a696f6e652064692052657469
    [Length: 22]
```

```
0000  02 00 00 00 45 00 00 32  79 53 00 00 80 11 00 00   ····E··2 yS······
0010  7f 00 00 01 7f 00 00 01  27 10 d6 65 00 1e da ee   ····· '··e····
0020  50 72 6f 67 72 61 6d 6d  61 7a 69 6f 6e 65 20 64   Programm azione d
0030  69 20 52 65 74 69                                  i Reti
```